

2. Semesters Rapport

Soul & Talk - Administrativt program til podcast planlægning.



Product Owner: Anne Wulff

Projektgruppe: Team A5

Lavet af: Jeppe Tungelund, Senan Salah, Zilas Andersen,
Peter Wulff, Nickolai Lock Jensen og Tore Jürgensen

Afleveringsfrist: 18. Maj 2026

GitHub link: https://github.com/PeterZimmertoft/Team_Soul_-_Talk-PodcastProgram

UCL Erhvervsakademi og Professionshøjskole

Antal anslag: 94555

Indholdsfortegnelse

1. Introduktion.....	1
1.1 Baggrund og foranalyse	1
1.2 Virksomhedsanalyse.....	1
1.2.1 Kort business case	2
1.2.2 Forretningsmodel-lærred	3
1.2.3 Forandringsledelse.....	5
1.3 Problemformulering	5
1.4 Afgrænsning	6
2. Kravspecifikation.....	7
2.1 Funktionelle krav.....	7
2.2 Ikke-funktionelle krav	8
2.3 Prioritering	9
2.4 Sporbarhed fra problem til løsning.....	10
3. Analyse	12
3.1 Use case-modellering	13
3.2 Fully dressed use cases.....	14
3.3 Domænemodel (DM)	20
3.6 System Operations Kontrakt (SOC).....	26
4. Design	27

4.1 Arkitektur og lagdeling	27
4.2 Pakkediagram	29
4.3 Designklassediagram.....	31
4.4 Sekvensdiagram	39
4.5 Databasedesign.....	43
4.5.1 Relational Databaseskema	43
4.5.2 UML DatabaseModel	45
5. Implementering	46
5.1 Implementeringsvalg	46
5.2 C#-kode	47
5.2.1 GuestViewModel.....	47
5.2.2 CreateGuestViewModel	48
5.2.2.1 SaveGuest() metode i CreateGuestViewModel.....	49
5.2.3 BasicGuestInformationIsValid : bool.....	50
5.2.4 CitizenInformationIsValid : bool.....	50
5.2.5 ValidationService : IValidationService	51
5.2.6 GuestRepository.Add	52
5.2.7 GuestRepository.GetById.....	53
5.2.8 GuestRepository.Update.....	54
5.2.9 GuestRepository.Delete	54

6. Test.....	56
6.1 Unit test	56
7. Projektstyring og arbejdsproces	57
7.1 Kommunikation.....	57
7.2 Versionsstyring og arbejdsmetode	58
7.3 Projektstyring	59
Sprints.....	59
8. Konklusion.....	62
9. Litteraturliste	63
10. Bilag.....	64
10.1 Bilag 1: GitHub repository	64
10.2 Bilag 2: SQL-Query til databasen	64
10.3 Bilag 3: MainView (hovedvinduet i programmet)	66
10.3.1 Bilag 3.1: GuestView (Gæstevinduet i programmet).....	66
10.4 Bilag 3.2: CreateGuestView (Opret gæsteprofil i programmet)	67
10.5 Bilag 4: Unit Test	68
10.6 Bilag 5: Komplet DCD for UC1	69

1. Introduktion

1.1 Baggrund og foranalyse

Soul & Talk er en enkeltmandsvirksomhed inden for social støtte og rådgivning.

Virksomheden arbejder med borgere, private personer og kommunale aktører, hvor der kan være behov for at holde styr på kontaktoplysninger, mødested, status og øvrige relevante informationer. Product Owner ønsker samtidig at anvende podcast som en ny kommunikationskanal, hvor erfaringer, viden og personlige fortællinger kan formidles.

I den nuværende arbejdsgang er oplysningerne ikke samlet i ét system. Det betyder, at overblikket over gæster, borgere og podcast-episoder kan blive afhængigt af manuelle noter og spredte dokumenter.

For at løse denne problemstilling udvikles et administrativt WPF-program (Windows Presentation Foundation). Systemet gør det muligt for PO at oprette profiler, registrere borgeroplysninger, administrere podcast-episoder og tilgå en samlet episodeoversigt. Rapporten følger en rød tråd, hvor virksomhedens behov omsættes fra indledende kravspecifikation og use cases til analyse, design og endelig implementering.

1.2 Virksomhedsanalyse

Virksomhedens kerneaktivitet er rådgivning og social støtte. Den nye podcast funktion kan ses som en strategisk udvidelse, fordi virksomheden får mulighed for at skabe indhold, samle erfaringer og bruge episoderne som en del af virksomhedens formidling. Systemets vigtigste forretningsmæssige værdi er et bedre overblik, færre manuelle fejl og lettere planlægning af episoder.

Interessenterne i projektet er primært PO, som er den daglige bruger af systemet. Derudover findes der indirekte interessenter som gæster, borgere og kommunale aktører, fordi deres oplysninger eller samarbejde kan indgå i arbejdsgangen.

I analysefasen overvejede vi at udarbejde detaljerede aktivitetsdiagrammer for at visualisere de komplekse arbejdsgange mellem PO, borgere og eksterne aktører. Vi valgte dog at udelade dette diagram, da vi ikke har en “as is”, fordi det er en nyopstartet podcast.

1.2.1 Kort business case

Den foretrukne løsning:

Den foretrukne løsning er et lokalt administrativt program med fokus på profiler og podcast-episoder. Alternativet er at fortsætte med manuelle noter og adskilte dokumenter, men det giver dårligere overblik, større risiko for fejl og mere tidsforbrug ved planlægning af podcast-episoder.

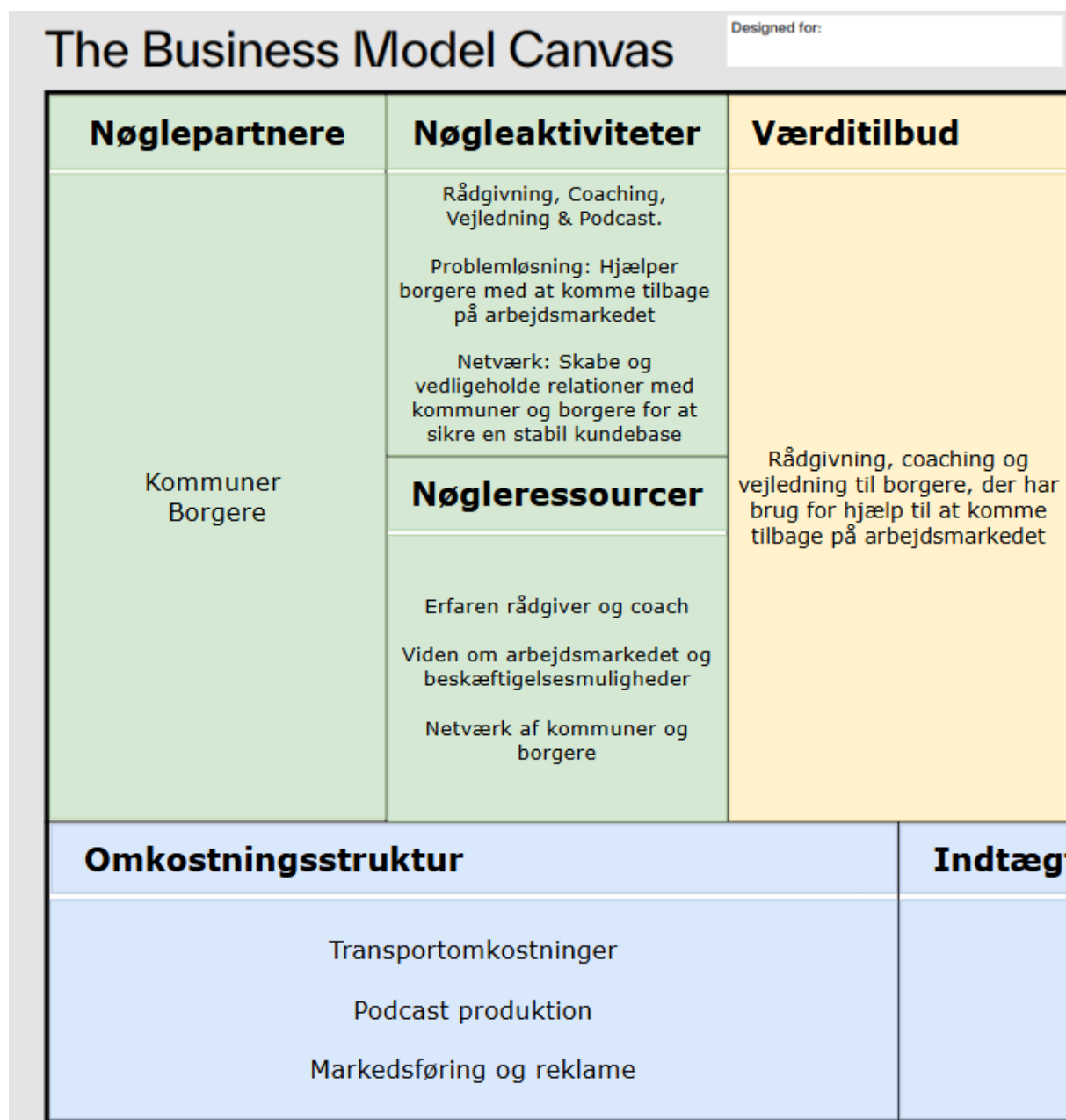
Forventet gevinst:

PO får ét samlet sted til at oprette gæster, registrere borgeroplysninger, planlægge episoder og se en episodeoversigt. Det reducerer behovet for manuelt arbejde på papir.

Risici:

Da systemet håndterer personoplysninger og CPR-lignende data, er projektet afgrænset til lokal datalagring og testdata. CPR-feltet valideres kun teknisk på format og anvendes ikke til opslag i offentlige registre.

1.2.2 Forretningsmodel-lærred



Figur 1 - Forretningsmodel-lærred (del 1) for Soul & Talk.

	Designed by:	Date:	Version:
Id	Kunderelationer	Kundesegmenter	
	Personlig kontakt Netværksrelationer	Parrådgivning Unge/udsatte borgere Voksne mennesker	
	Kanaler		
aching og jere, der har l at komme lsmarkedet	Sociale medier Podcast "Soul and Talk" Fysisk kontakt Telefoniske samtaler		
Indtægtsstrømme			
Private indtægter Kommunale indtægter			

Figur 2 - Forretningsmodel-lærred (del 2) for Soul & Talk.

Der er blevet udarbejdet en forretningsmodel-lærred (FML) for Soul & Talks virksomhed. Dette er gjort for at få et dybere indblik i PO's forretning, indtægtsstrømme og kunderelationer. Denne analyse giver et dybere indblik på deres samarbejdspartnere, som primært ligger mellem borgere og kommuner. Deres nøgleaktiviteter er primært blevet inddelt i 3 forskellige områder. Rådgivning, som består af coaching, podcast og vejledning. Problemløsning, som går ud på at få borgere tilbage på arbejdsmarkedet, og til sidst Netværk, som handler om at knytte et tæt bånd, både mellem borgere og kommuner. Det, der er mest interessant for vores projekt, er at få et indblik i, hvordan deres podcast påvirker virksomheden. En af PO's omkostningspunkter peger mod podcast produktion. Dette betyder ikke nødvendigvis at PO bruger en masse penge på det, men derimod at der bliver brugt en masse tid og energi på dette punkt. Dette vil vi gerne reducere med vores løsning, ved at give et større overblik, samt en langt nemmere og optimeret løsning for Soul & Talk. Derudover kunne en fremtidig version af virksomhedens FML, have deres podcast under værditilbud. Selvom podcasten ikke nødvendigvis skaber direkte indtægt, kan den skabe værdi ved at give indsigt i borgeres erfaringer og udfordringer. Derfor kan podcasten i en fremtidig version af forretningsmodellen også ses som en del af virksomhedens værditilbud.

1.2.3 Forandringsledelse

Forandringsledelse spiller en rolle i projektet, fordi systemet ændrer den måde, PO arbejder på. I stedet for at bruge spredte noter og dokumenter samles oplysningerne i ét program. Derfor skal løsningen være enkel og nem at forstå, så den nye arbejdsgang bliver lettere at tage i brug.

1.3 Problemformulering

Soul & Talk mangler et samlet system til at håndtere de forskellige aktørtyper og de oplysninger, der knytter sig til dem. Der er behov for at kunne oprette og vedligeholde profiler på personer, som kan indgå i en podcast-episode. Nogle profiler er almindelige

gæster, mens andre også indeholder borgeroplysninger, fordi de er knyttet til virksomhedens sociale støtte- og rådgivningsarbejde.

Derudover mangler virksomheden et værktøj til at skabe overblik over oprettede podcast-episoder. PO har behov for at kunne se en samlet liste over episoder og vælge en bestemt episode for at få vist dens oplysninger samme sted i systemet. Det omfatter eksempelvis titel, dato, tidspunkt, status, mødested, noter og eventuelle tilknyttede gæster.

Problemformulering

Hvordan kan der udvikles et administrativt WPF-program til Soul & Talk, som samler håndteringen af borger- og gæsteprofiler, øvrige relevante borgeroplysninger og podcast-episoder, så PO får et bedre overblik og en mere ensartet arbejdsgang.

1.4 Afgrænsning

Afgrænsningen er vigtig, fordi projektet skal kunne gennemføres inden for rammerne af 2. Semester. Fokus er derfor på et program, der viser forståelse og analyse, design, lagdeling, database og implementering, frem for et komplet produktionsklart system.

Vores løsning er et WPF-program, da det integrerer læringsmaterialet i et passende omfang. Systemet håndterer profiler, borgeroplysninger og podcast-episoder. Løsningen håndterer ikke publicering, lydfiler, kalenderintegration, MitID eller CPR-validering ift. offentlige registre. CPR-feltet anvendes kun som testfelt og valideres teknisk på format og længde, eksempelvis DDMMYY-XXXX. Systemet kontrollerer ikke, om CPR-nummeret findes i offentlige registre.

2. Kravspecifikation

Kravspecifikationen omsætter virksomhedens behov til konkrete krav til systemet. De funktionelle krav beskriver, hvad systemet skal kunne, mens de ikke-funktionelle krav beskriver, hvordan systemet bør fungere og opbygges.

2.1 Funktionelle krav

ID	Krav	Beskrivelse
FK1	Opret profil	Systemet skal kunne oprette en profil med navn, telefonnummer og e-mail.
FK2	Registrer borgeroplysninger	Systemet skal kunne tilføje borgeroplysninger til en profil, når profilen også er en borger.
FK3	Opdater profil	Systemet skal kunne redigere eksisterende gæste- og borgeroplysninger.
FK4	Opret podcast-episode	Systemet skal kunne oprette en podcast-episode med titel, dato, status, mødested og note.
FK5	Tilknyt gæster	Systemet skal kunne tilknytte nul, en eller flere gæster til en podcast-episode.

FK6	Vis episodeoversigt	Systemet skal kunne vise en oversigt over oprettede episoder og detaljer for den valgte episode.
FK7	Valider input	Systemet skal kontrollere obligatoriske felter og formatkrav, herunder at CPR-feltet valideres kun teknisk på format og længde, eksempelvis DDMMYY-XXXX. Systemet kontrollerer ikke, om CPR-nummeret findes i offentlige registre.
FK8	Slet profil	Systemet skal kunne slette en eksisterende gæsteprofil og eventuelle tilknyttede borgeroplysninger.
FK9	Slet podcast-episode	Systemet skal kunne slette en valgt podcast-episode efter brugerbekræftelse.

2.2 Ikke-funktionelle krav

ID	Kravtype	Beskrivelse
IFK1	Brugervenlighed	Programmet skal være enkelt at bruge for én primær bruger, uden teknisk baggrund.

IFK2	Arkitektonisk struktureret	Koden skal være opdelt i lag, så View, ViewModel, Model, Services og Persistence kan ændres uden nødvendig kobling.
IFK3	Datasikkerhed	Systemet skal kun gemme oplysninger, der er nødvendige for oprettelse af profiler og podcast-episoder. CPR-relaterede oplysninger behandles som følsomme testdata i projektet og anvendes ikke til eksterne opslag.
IFK4	Testbarhed	Repository-, service- og ViewModel-logik skal kunne testes uden direkte afhængighed af brugergrænsefladen.
IFK5	Sporbarhed	Use cases, diagrammer, designklasser, database og kode skal kunne spores til samme funktionelle behov.

2.3 Prioritering

Prioriteringen tager udgangspunkt i PO's vigtigste arbejdsgange. Først skal systemet kunne oprette profiler, da episoder og borgeroplysninger afhænger af, at personer kan registreres. Derefter prioriteres oprettelse af podcast-episoder og redigering af eksisterende oplysninger. Episodeoversigten er også central, fordi den direkte understøtter virksomhedens behov for overblik.

s	Use case	Use case-navn	Beskrivelse	Estimat
1	UC1	Opret profil til podcast episode	En profil oprettes i systemet, så den senere kan tilknyttes en podcast-episode.	Stor
2	UC2	Planlæg podcast-episode	PO kan oprette en podcast-episode og eventuelt tilknytte eksisterende gæster.	Stor
3	UC3	Opdater borgerprofil	PO kan opdatere en allerede eksisterende profil og eventuelle borgeroplysninger.	Stor
4	UC4	Se episodeoversigt	PO kan se en samlet oversigt over episoder og detaljer for en valgt episode.	Lille
5	UC5	Slet podcast-episode	Systemet skal kunne slette en valgt podcast-episode efter brugerbekræftelse.	Mellem

2.4 Sporbarhed fra problem til løsning

For at bevare den røde tråd, kobles virksomhedens problem til kravene, use cases, design og implementering. Tabellen viser, hvordan rapportens dele hænger sammen:

Virksomhedsbehov	Krav	Use case	Design/Implementering
Samle personoplysninger ét sted	FK1, FK2, FK3, FK7	UC1, UC3	Guest og Citizen anvendes som centrale domæneklasser. GuestViewModel og CreateGuestViewModel håndterer visning, oprettelse og redigering af profiler. Dataadgang sker gennem IGuestRepository og ICitizenRepository. ValidationService bruges til inputvalidering, mens IMessageService bruges til brugerbeskeder.
Kunne planlægge og registrere podcast-episoder	FK4, FK5	UC2	PodcastEpisode anvendes som domæneklasse for episoder. CreatePodcastEpisodeViewModel håndterer oprettelse af episoder og tilknytning af gæster. Mange-til-mange relationen mellem podcast-episoder og gæster håndteres i databasen gennem koblingstabellen PodcastEpisodeGuests og i koden gennem IPodcastEpisodeRepository.

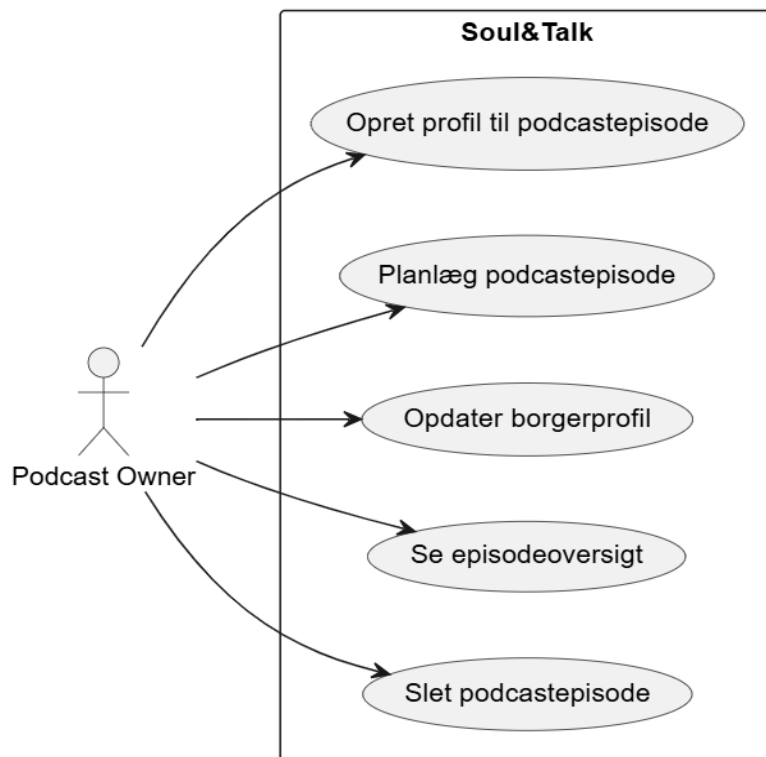
Skabe overblik over episoder	FK6	UC4	PodcastEpisodeView og PodcastEpisodeViewModel viser en samlet episodeoversigt og detaljer for den valgte episode. Tilknyttede gæster hentes gennem IPodcastEpisodeRepository, så PO kan se relevante oplysninger samme sted.
Undgå tæt koblet kode	NF2, NF4, NF5	Alle use cases	Systemet er opbygget med MVVM, lagdeling, services, repository-interfaces og konkrete command-klasser. Views er adskilt fra logik gennem databinding, ViewModels bruger interfaces frem for konkrete repositories, og tværgående funktionalitet som navigation, beskeder og validering håndteres gennem services.

3. Analyse

Analyseafsnittet beskriver vores system, set fra brugerens og domænets perspektiv. Først beskriver vi vores use cases, derefter de centrale begreber i domænemodellen, og til sidst vises systemoperationer gennem SSD og systemoperationskontrakt.

3.1 Use case-modellering

Use cases er brug til at beskrive samspillet mellem PO og vores system. Formålet er at sikre at funktionaliteten tager udgangspunkt i brugerbehovet



Figur 3 - Use case-diagram for Soul & Talk.¹

Use Case #	Use Case Navn
UC1	Opret profil til podcastepisode
UC2	Planlæg podcastepisode
UC3	Opdater borgerprofil
UC4	Se episodeoversigt
UC5	Slet podcastepisode

Figur 4 - Oversigt over use cases.

¹ Tegnet i plantUML

3.2 Fully dressed use cases

UC1: Opret profil til podcast episode

Aktør:	Podcast Owner
Mål:	At oprette en ny profil med de nødvendige oplysninger
Level:	User goal
Preconditions:	Systemet er startet, og PO har åbnet funktionen til oprettelse af en ny profil.
Postconditions:	Profilen er oprettet i systemet og kan senere tilknyttes en podcast-episode. Hvis borgeroplysninger er indtastet, gemmes disse og tilknyttes profilen.

Hovedscenarie:

1. PO anmoder om at oprette en ny profil.
2. Systemet præsenterer en skabelon til indtastning af profiloplysninger.
3. PO angiver profilens basisoplysninger, herunder navn, telefon og e-mail.
4. PO indtaster eventuelt borgeroplysninger for profilen.
5. Systemet validerer de indtastede oplysninger.
6. Systemet kontrollerer, om profilen allerede findes i systemet.

Alternative forløb:

5A: De indtastede oplysninger er ugyldige. Systemet viser en fejlmeddelelse til PO og afventer rettelse.

6A: Profilen findes allerede i systemet. Systemet informerer PO om, at profilen allerede eksisterer.

UC2: Planlæg podcast-episode

Aktør:	Podcast Owner
Mål:	At oprette en podcast-episode i systemet med relevante oplysninger og eventuelt tilknytte gæster
Level:	User goal
Preconditions:	Systemet er startet, og PO har åbnet funktionen til oprettelse af en podcast-episode.
Postconditions:	Podcast-episoden er oprettet i systemet. Eventuelle valgte gæster er tilknyttet episoden. Hvis ingen gæster er valgt, er episoden stadig oprettet og kan opdateres senere.

Hovedscenarie:

1. PO anmoder om at oprette en ny podcast-episode.
2. Systemet viser en skabelon til indtastning af episodeoplysninger.
3. PO indtaster episode oplysninger, herunder titel, dato, varighed, status, mødested og note.
4. PO vælger eventuelt en eller flere eksisterende gæster, der skal tilknyttes episoden.
5. Systemet opretter podcast-episoden.
6. Systemet tilknytter de valgte gæster til episoden, hvis nogen er valgt.
7. Systemet bekræfter, at podcast-episoden er oprettet.

Alternative forløb:

3A: Obligatoriske episode oplysninger mangler. Systemet viser en fejlmeddelelse.

4A: PO vælger ingen gæster. Systemet opretter episoden uden tilknyttede gæster.

UC3: Rediger eksisterende profil til podcast-episode

Aktør:	Podcast Owner
Mål:	At opdatere oplysninger på en eksisterende profil i systemet.
Level:	User goal
Preconditions:	Profilens basisoplysninger og eventuelle borgeroplysninger er gemt i systemet.
Postconditions:	Profilens oplysninger er opdateret og gemt i systemet.

Hovedscenarie:

1. PO vælger en eksisterende gæst fra gæstelisten.
2. PO vælger at redigere gæsten.
3. Systemet viser de nuværende oplysninger.
4. PO ændrer basisoplysninger og eventuelt borgeroplysninger.
5. Systemet validerer oplysningerne.
6. Systemet gemmer ændringerne.
7. Systemet bekræfter, at profilen er opdateret.

Alternative forløb:

5A: CPR-nummeret findes allerede på en anden borger. Systemet viser fejlbesked.

UC4: Se episodeoversigt

Denne use case beskriver, hvordan PO åbner podcastoversigten i systemet, ser en liste over episoder og vælger en episode for at se dens oplysninger.

Aktør:	Podcast Owner
Mål:	At få overblik over oprettede podcast-episoder og se oplysninger om en valgt episode.
Level:	User goal
Preconditions:	Systemet er startet.

Postconditions:	Systemet viser episodeoversigten og eventuelt detaljer for den valgte episode.
------------------------	--

Hovedscenarie:

1. PO åbner episodeoversigten.
2. Systemet viser en liste over oprettede episoder.
3. PO vælger en episode fra listen.
4. Systemet viser oplysninger om den valgte episode.
5. PO gennemser episodeoplysningerne.

Alternative forløb:

2A:Hvis der ikke findes episoder, viser systemet en tom oversigt.

3A: Hvis PO ikke vælger en episode, vises listen fortsat uden detaljer.

UC5: Slet podcast-episode

Denne use case beskriver, hvordan PO åbner podcastoversigten i systemet, ser en liste over episoder, og vælger en episode for at slette den.

Aktør:	Podcast Owner
Mål:	At fjerne en oprettet podcast-episode fra systemet
Level:	User goal

Preconditions:	Podcast-episoden findes i systemet
Postconditions:	Podcast-episoden og dens gæstetilknytninger er slettet

Hovedscenarie:

1. PO vælger en podcast-episode.
2. PO trykker på slet episode.
3. Systemet beder om bekræftelse.
4. PO bekræfter sletningen.
5. Systemet sletter podcast-episoden og dens gæstetilknytninger.
6. Systemet opdaterer episodeoversigten.

Kvalitetskriterier for use cases:

Use casene er udarbejdet som fully dressed use cases og fungerer som grundlag for den videre analyse og design af systemet. De er skrevet på user-goal niveau, fordi de beskriver mål, som Podcast Owner ønsker at opnå i systemet.

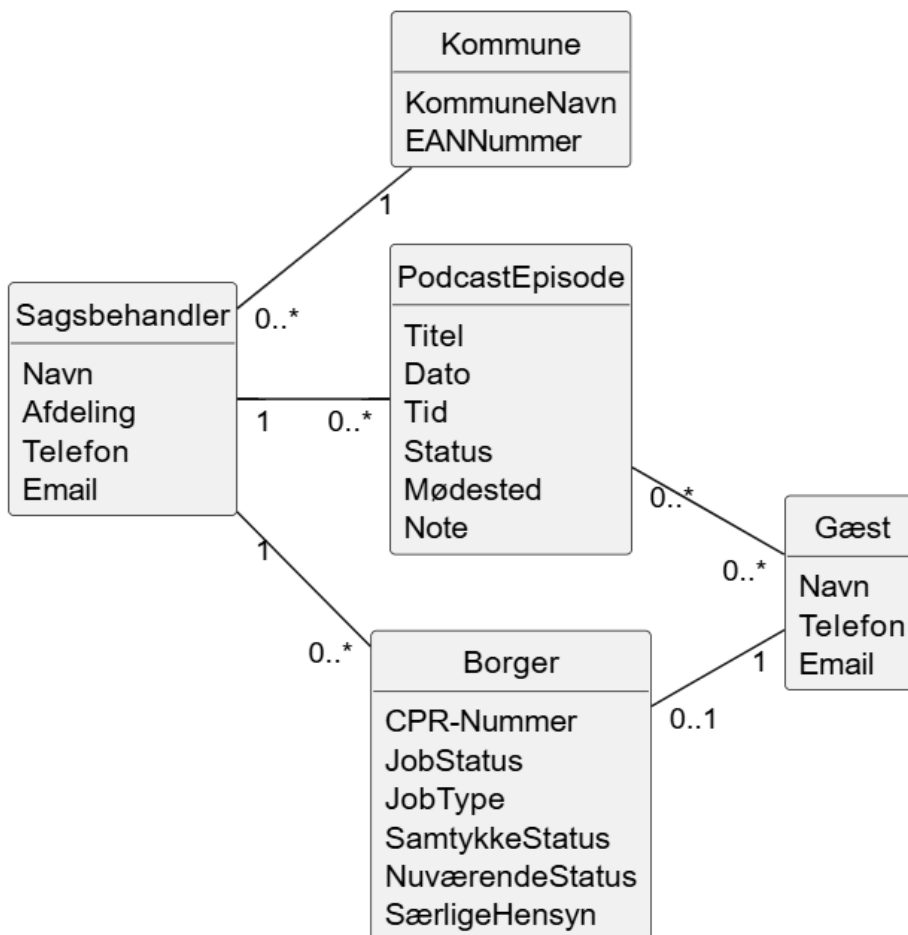
For at sikre kvaliteten af use casene er de vurderet ud fra Boss-testen, Size-testen og EBP-testen. Boss-testen er anvendt til at vurdere, om use casen skaber en værdi, som er relevant for virksomheden. Size-testen er anvendt til at sikre, at use casene hverken er for små eller for omfattende. EBP-testen er anvendt til at vurdere, om hver use case afsluttes med et meningsfuldt resultat for aktøren og systemet. Eksempelvis afsluttes UC1 med, at en profil er oprettet, mens UC2 afsluttes med, at en podcast-episode er gemt og eventuelt tilknyttet én eller flere gæster.

Sporbarheden sikres ved, at begreber og handlinger fra use casene videreføres i resten af rapporten. UC1 danner grundlag for arbejdet med Guest og Citizen, UC2 danner grundlag

for PodcastEpisode og tilknytning af gæster, UC3 understøtter redigering af eksisterende profiler, og UC4 understøtter episodeoversigten.

3.3 Domænemodel (DM)

Vores domænemodel bygger videre på problemformuleringen. Den viser de begreber, PO arbejder med: Gæster, borgere, podcast-episoder, kommuner og sagsbehandlere. Kommunen har vi dog valgt ikke at inkludere i vores implementering, da dette ikke giver mening for systemets nuværende afgrænsning. Hvis vi havde haft mere tid, kunne vi have udarbejdet specifikke use cases til at håndtere kommunikationen med kommunen.



Figur 5 - Domænemodel for Soul & Talk.²

² Tegnet i PlantUML

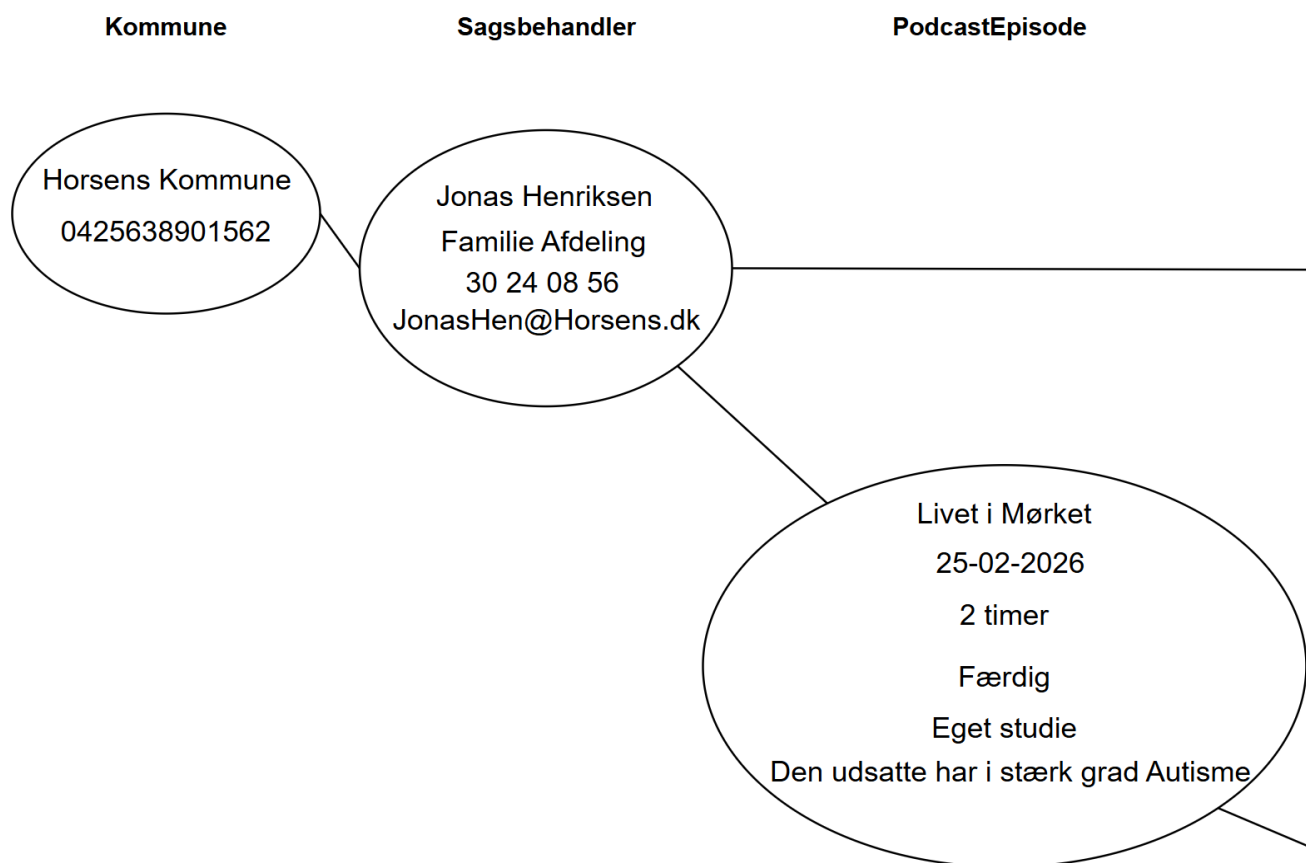
Modellen viser blandt andet, at en gæst kan være tilknyttet nul til mange podcast-episoder, og at en podcast-episode kan oprettes med nul til mange gæster. Det passer til kravet om, at episoder skal kunne oprettes, selvom gæster først vælges senere. Borgeroplysninger er knyttet til de profiler, hvor der er behov for mere end almindelige kontaktoplysninger. Kommune indgår i domænemodellen som en tydelig konceptuel klasse, men er ikke implementeret som en selvstændig klasse i den nuværende version af systemet.

Kvalitetskriterier for domænemodellen:

Klasserne er navngivet med navneord, multipliciteterne viser associationer mellem klasserne, og terminologien afspejler virksomhedens arbejdsgang og sprog. Modellen holdes konceptuel. Domænemodellen fokuserer på begreber og relationer og de forskellige klasser er opdelt i to felter - titel og attributter.

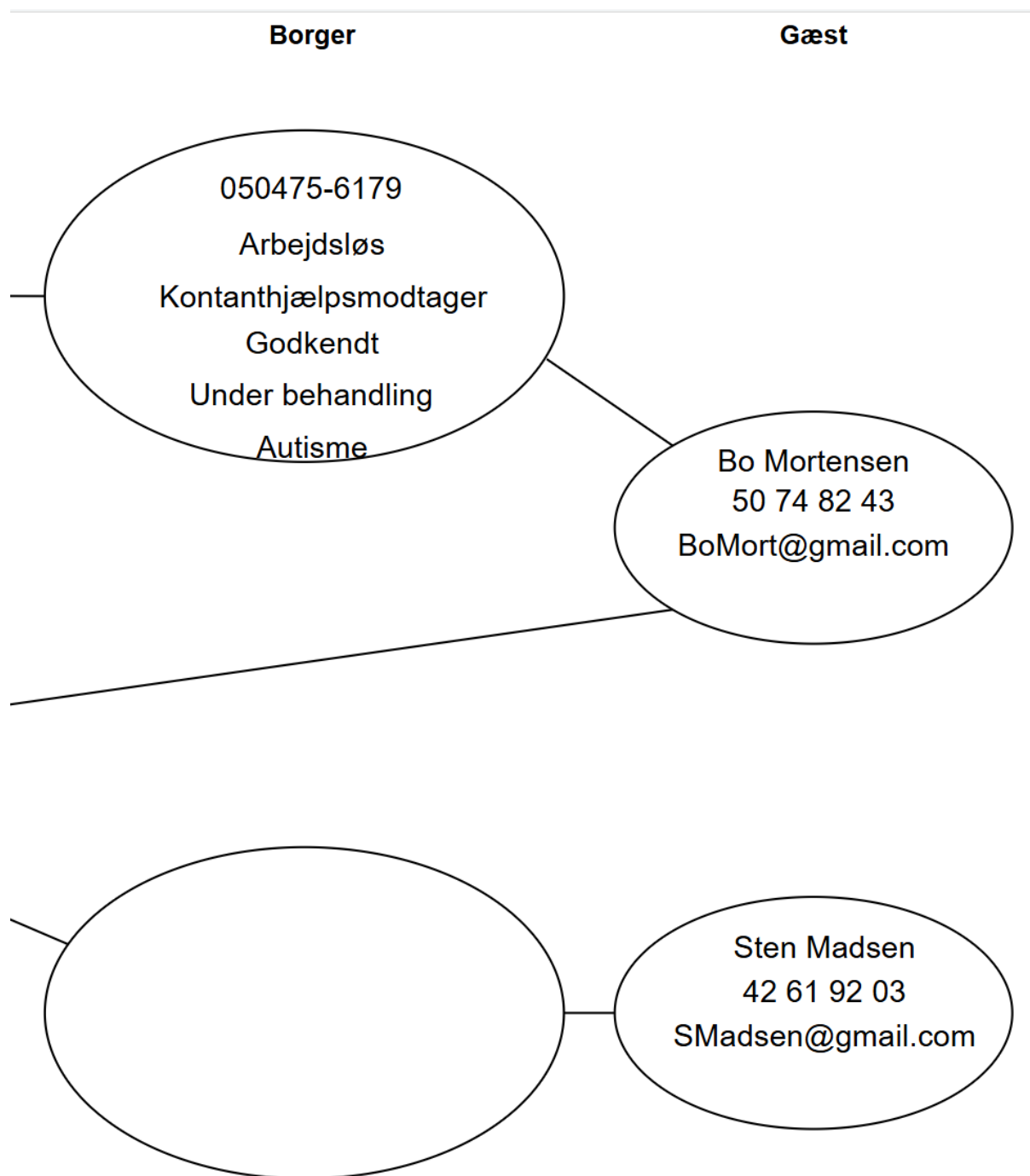
3.4 Objektmodel (OM)

Vores objektmodel viser et konkret eksempel på, hvordan objekter kan hænge sammen i systemet. I vores objektmodel har vi fx “PodcastEpisode”, og det objekt kommer fra vores “PodcastEpisode”-klasse fra domænemodellen.



Figur 6 - Objektmodel (del 1) for Soul & Talk.³

³ Tegnet i Draw.io



Figur 7 - Objektmodel (del 2) for Soul & Talk.⁴

Objektmodellen understøtter den røde tråd, fordi den viser den arbejdsgang, der beskrives i kravene: en profil kan have borgeroplysninger, og en podcast-episode kan have tilknyttede personer og relevante noter.

⁴ Tegnet i Draw.io

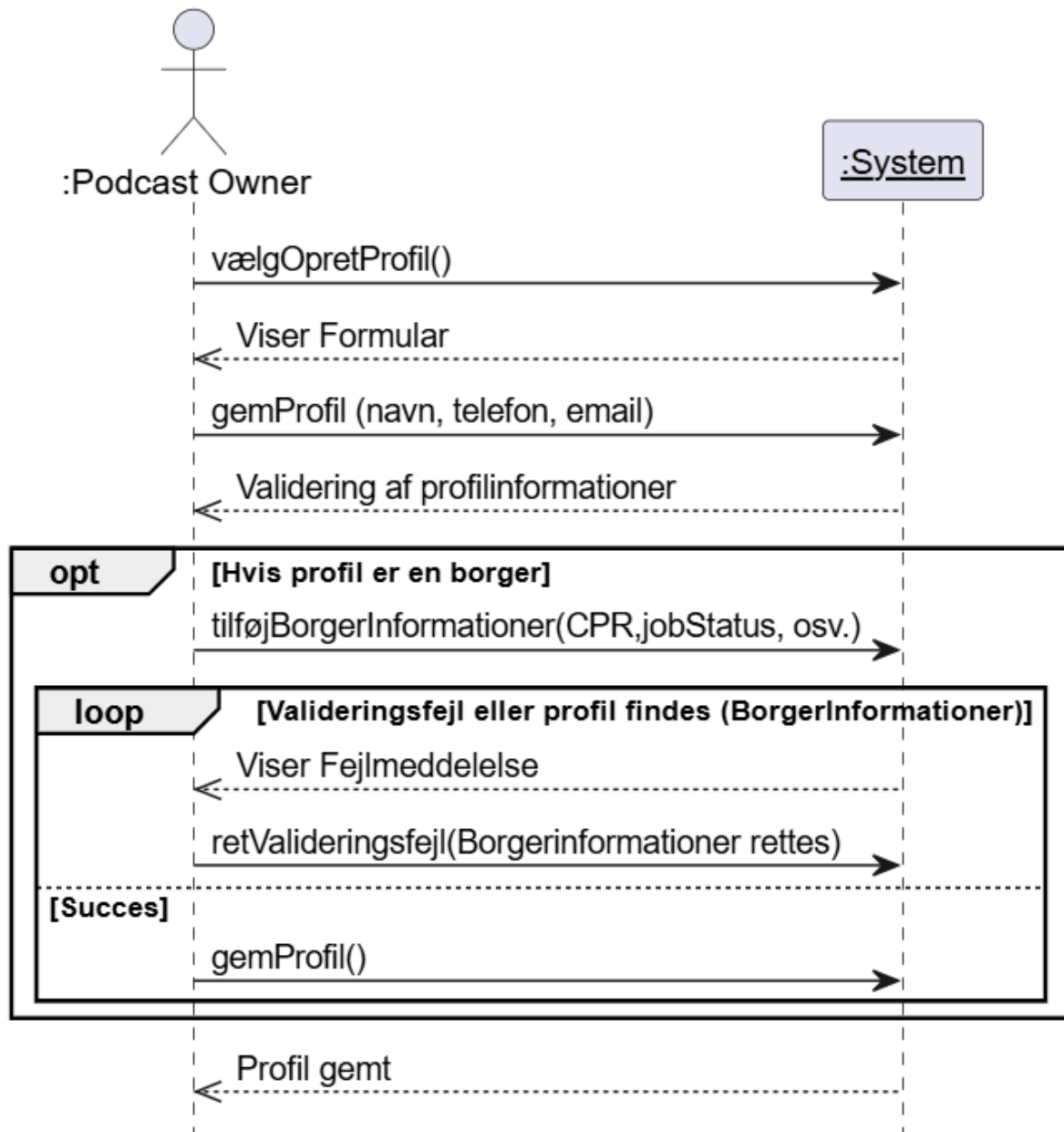
Kvalitetskriterier for objektmodellen:

Forbindelserne mellem objekterne vises med streger, og objekterne er forbundet korrekt i forhold til klassernes relationer fra domænemodellen. Hvert objekt er ellipseformet, og de konkrete værdier fra objekterne er et eksempel på data. Objektmodellen viser et konkret eksempel på de associationer, der er beskrevet i domænemodellen.

3.5 Systemsekvensdiagram (SSD)

Systemsekvensdiagrammet viser systemhændelser set udefra i et black-box perspektiv. Det betyder, at diagrammet ikke viser interne systemhændelser, kun hændelser aktøren sender til system, og hvilke svar systemet giver tilbage:

UC1: Opret profil til podcast episode



Figur 8 - SSD for Soul & Talk.⁵

⁵ Tegnet i PlantUML

Kvalitetskriterier for systemsekvensdiagrammet:

I vores SSD har vi taget udgangspunkt i UC1, som er repræsenteret som en understreget overskrift. Aktøren repræsenteres med en tændstik figur, hvor de systemhændelser, der opstår fra aktørens handlinger hen til systemet, er fuldt optrukken med en lukket pil og er skrevet i camelCase samt er et metodekald. Systemrespons, som er de pile som kommer tilbage fra system til aktøren, er illustreret med en stiptet, åben pil. Livslinjerne er stiplede linjer, som viser levetiden for SSD'et.

3.6 System Operations Kontrakt (SOC)

Vores SOC viser operationen "OpretProfil" fra vores UC1, hvilket kaldes for dens cross reference. Her viser den, hvilke attributter der er tilknyttet: navn, telefon og email. Derefter definerer vi preconditions som er forudsætningerne der skal være opfyldt for at operationen kan udføres. Disse er, at systemet er opstartet, samt at PO har valgt at oprette en ny profil i systemet. Derefter vises de postconditions som denne operation skaber, dette er tilstanden af systemet efter operationen finder sted. Der sker en instance creation, da et nyt "Guest" objekt bliver oprettet i systemet, hvis borgeroplysninger er udfyldt under oprettelsen, bliver et "Citizen" objekt oprettet og tilknyttet instansen af "Guest"-objektet. En association dannes, hvis der oprettes borgeroplysninger, fordi Citizen tilknyttes den oprettede Guest. Til sidst bliver borgeren gemt, hvilket er en attribute modification. Til slut viser systemet en visuel bekræftelse af, at profilen er blevet gemt, hvilket er SOC'ens output.

Systemoperationskontrakter (SOC) for Soul&Talk

SOC_OpretProfil	
Operation	OpretProfil(navn, telefon, email)
Cross_reference	UC1: Opret profil til podcast episode
Precondition	Systemet er startet, og PO har valgt at oprette en ny profil
Postcondition	En ny Guest blev oprettet (instance creation). Hvis borgeroplysninger var udfyldt, blev en Citizen oprettet og tilknyttet den oprettede gæst. Borgeren blev associeret med podcasten (association formed) Borger gemt (attribute modification)
Output	Systemet viste en bekræftelse på, at profilen blev gemt.

Figur 9 - Systemoperationskontrakt for UC1.⁶

Kvalitetskriterier for system operationskontrakten:

SOC'en er skrevet i datid og stemmer overens med SSD'et og dens cross reference fra use casen. Der er fokus på pre- og postconditions, da det er de to vigtigste punkter.

4. Design

4.1 Arkitektur og lagdeling

Systemet er designet som en WPF-applikation med Model-View-ViewModel (MVVM). Formålet med MVVM er at adskille brugergrænsefladen fra præsentation-logikken og data-adgang. Det giver bedre testbarhed og gør det lettere at ændre views uden at ændre domæne- og persistence laget.

Applikationen er opdelt i flere lag, som har hver deres respektive ansvar. Præsentationslaget består af views skrevet i XAML, såsom *GuestView* og *PodcastEpisodeView*, der håndterer

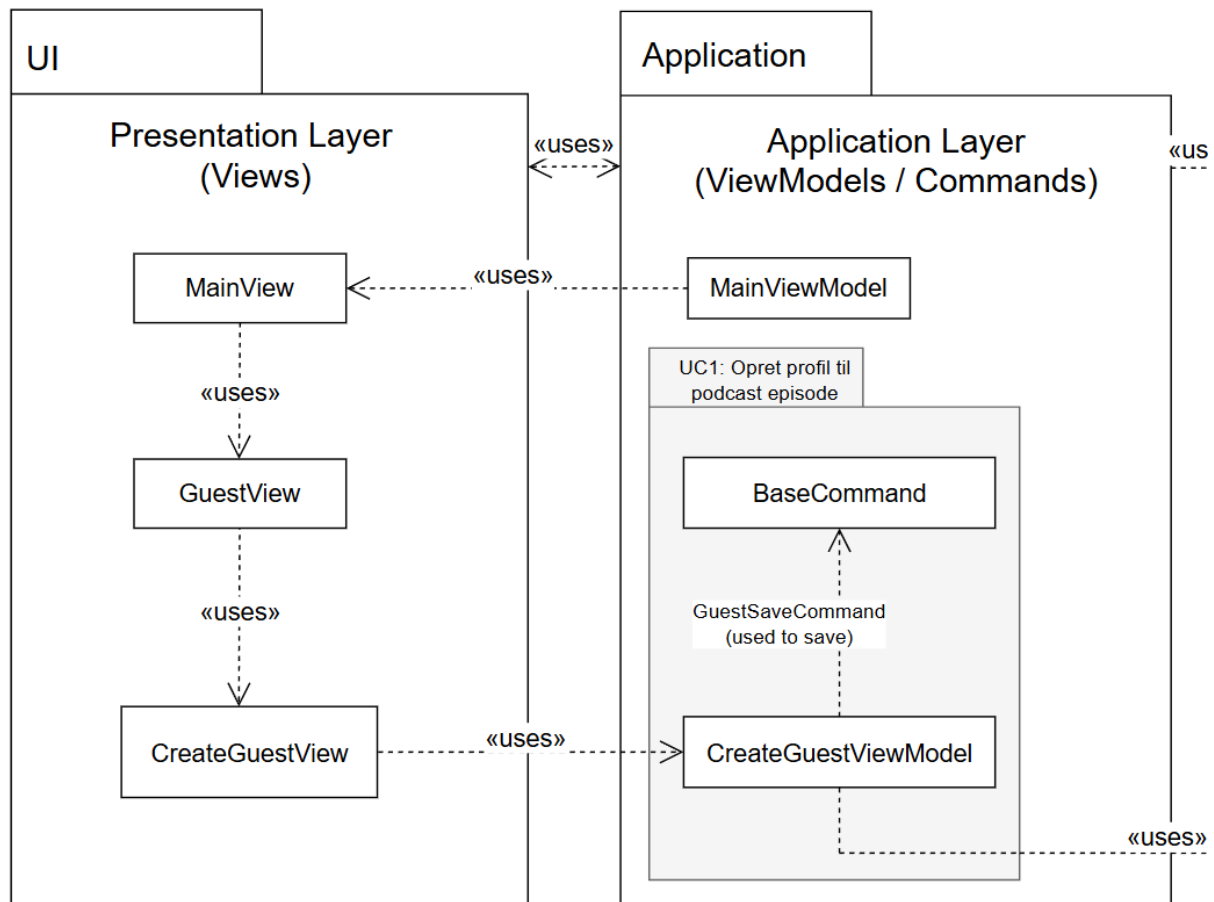
⁶ Lavet i PlantUML

brugergrænsefladen og interaktionen med brugeren. Applikationslaget indeholder ViewModels og commands, eksempelvis *MainViewModel*, *GuestViewModel* og *CreateGuestViewModel*, som styrer logikken bag brugergrænsefladen, og binder data mellem views, og resten af systemet. Domæne- eller Model-laget, rummer de centrale forretningsobjekter som *Guest*, *Citizen* og *PodcastEpisode*, der repræsenterer applikationens kernebegreber. Persistenslaget består af repositories, som indkapsler adgangen til databasen og håndterer SQL-forespørgsler. Endelig dannes det tekniske servicelag af en MS SQL Server-database, hvor projektets tabeller og relationer er defineret og lagret.

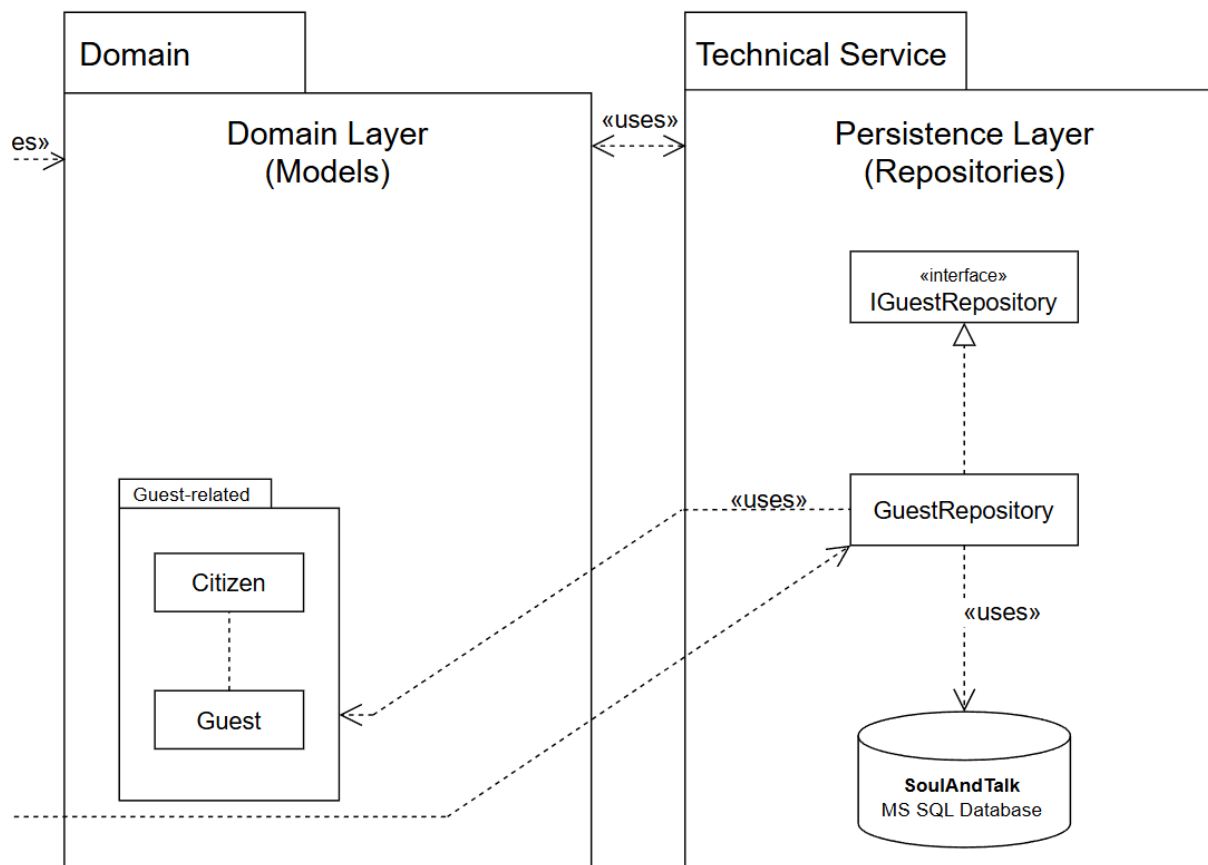
Derudover anvendes services til tværgående funktionalitet i systemet, som navigation, brugerbeskeder, validering og valg af gæster i et dialogvindue. Disse services gør ViewModels mindre afhængige af konkrete WPF-komponenter.

4.2 Pakkediagram

Pakkediagrammet viser de vigtigste afhængigheder mellem lagene. Afhængigheden går fra View til ViewModel, videre til domæne og persistence. Databinding mellem View og ViewModel er ikke vist, men der er en tovejsrelation, mens databaseadgang er isoleret i persistence-laget:



Figur 10 - Pakkediagram (del 1) for UC1 og de centrale lag.



Figur 11 - Pakkediagram (del 2) for UC1 og de centrale lag.

Kvalitetskriterier pakkediagrammet:

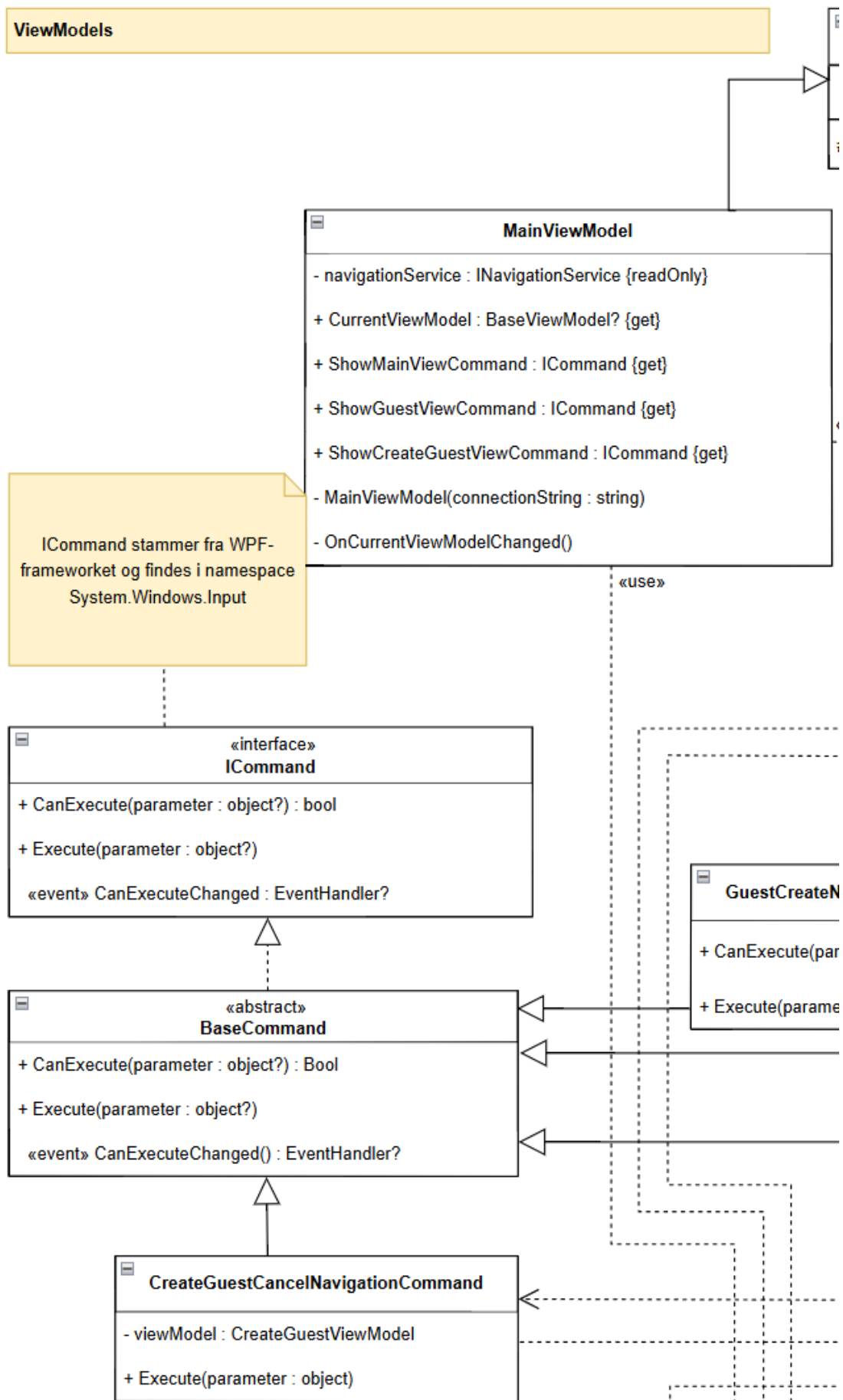
Klasser, der kun indgår i en use case, kan med fordel placeres i en separat pakke navngivet efter use casen for bedre overblik. Afhængigheder følger som udgangspunkt en retning fra UI til applikation til domæne til technical services. Pakkerne skal visualiseres som rektangler med pakkenavn øverst og indeholder klasser som mindre rektangler. Stiplede pile angiver afhængigheder.

4.3 Designklassediagram

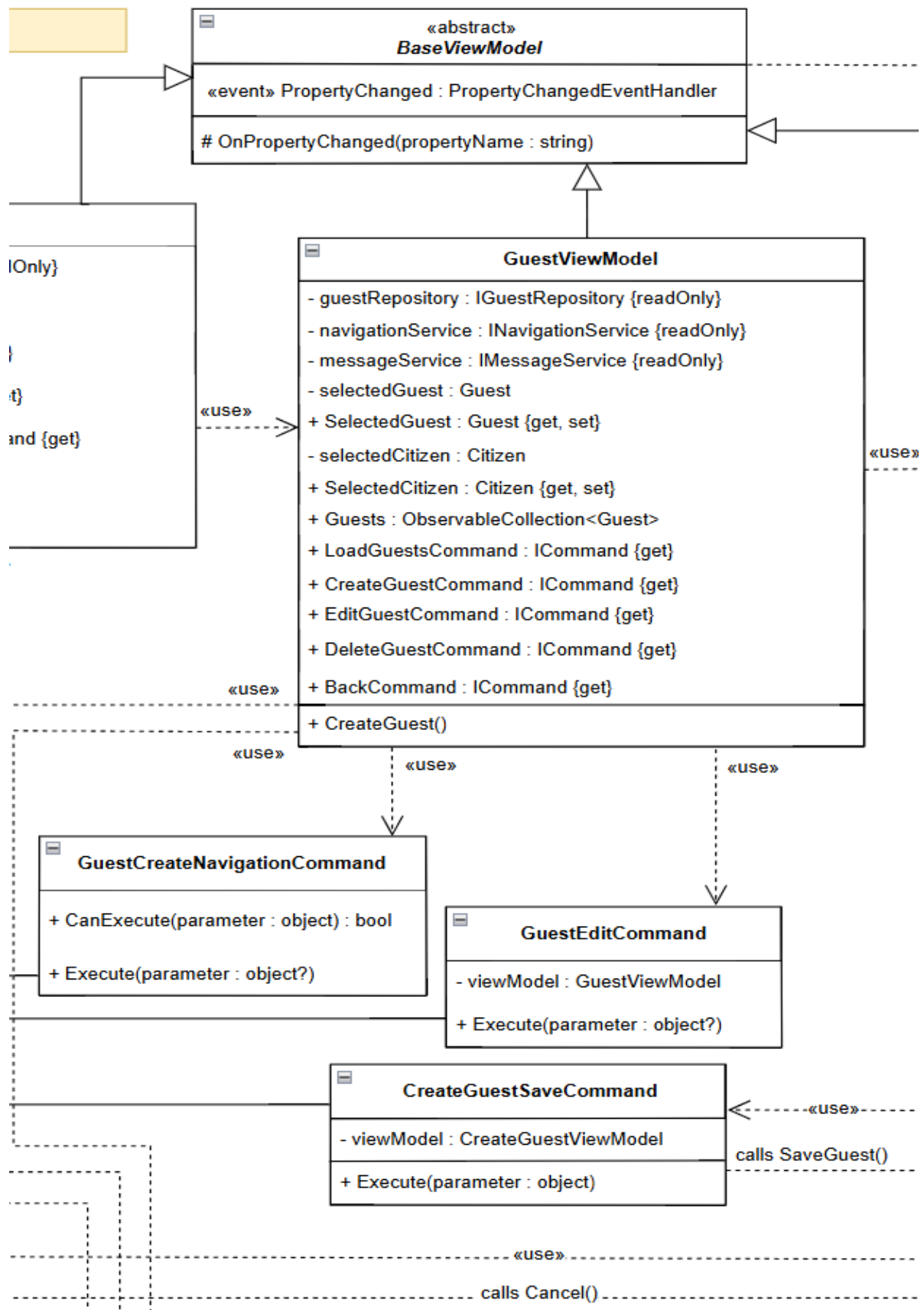
Vores DCD bruger vi til at definere det præcise ansvar for klasser som `CreateGuestViewModel`, `GuestRepository` og domæneobjektet `Guest`. For hver klasse har vi fastlagt specifikke attributter og operationer med tilhørende datatyper, såsom `AddGuest (Guest)` og `ProfileExists(string, int, string)`. Dette sikrer, at der ikke opstår tvivl under implementeringen om hvilke data, der skal sendes mellem komponenterne, eller hvilke returtyper en operation skal levere.

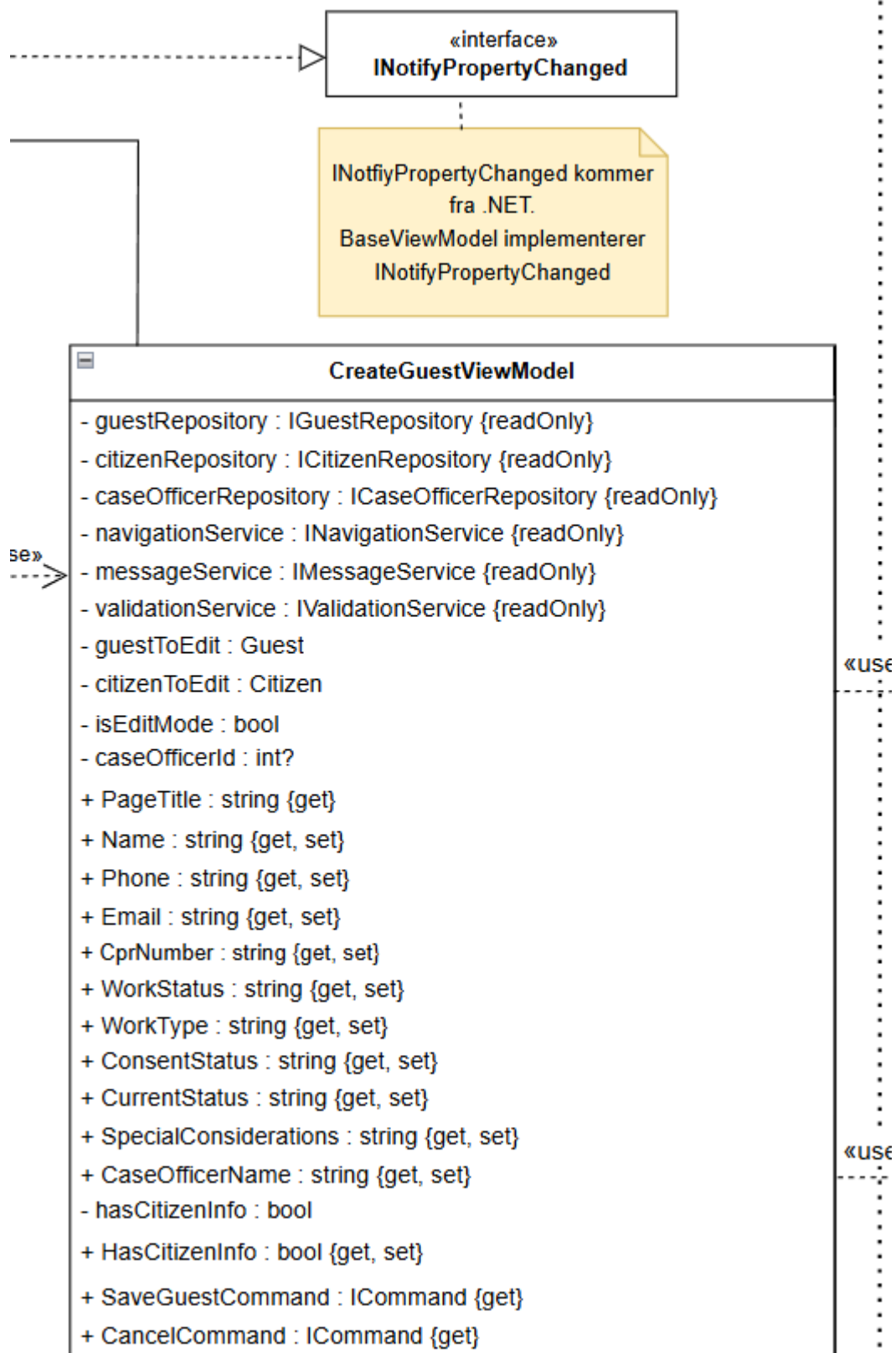
Relationerne i diagrammet definerer, hvordan objekterne har kendskab til hinanden i praksis. Vi bruger navigabiliteten i diagrammet til at styre, at vores ViewModels kan tilgå Repositories, men ikke omvendt, hvilket opretholder vores arkitektoniske lagdeling. Kardinaliteterne i DCD'et dikterer samtidig, hvordan vores samlinger af data skal struktureres i koden. Ved at følge dette diagram sikrer vi, at systemet bliver bygget ensartet, og at alle forretningsregler fra vores use case er understøttet af den tekniske infrastruktur.

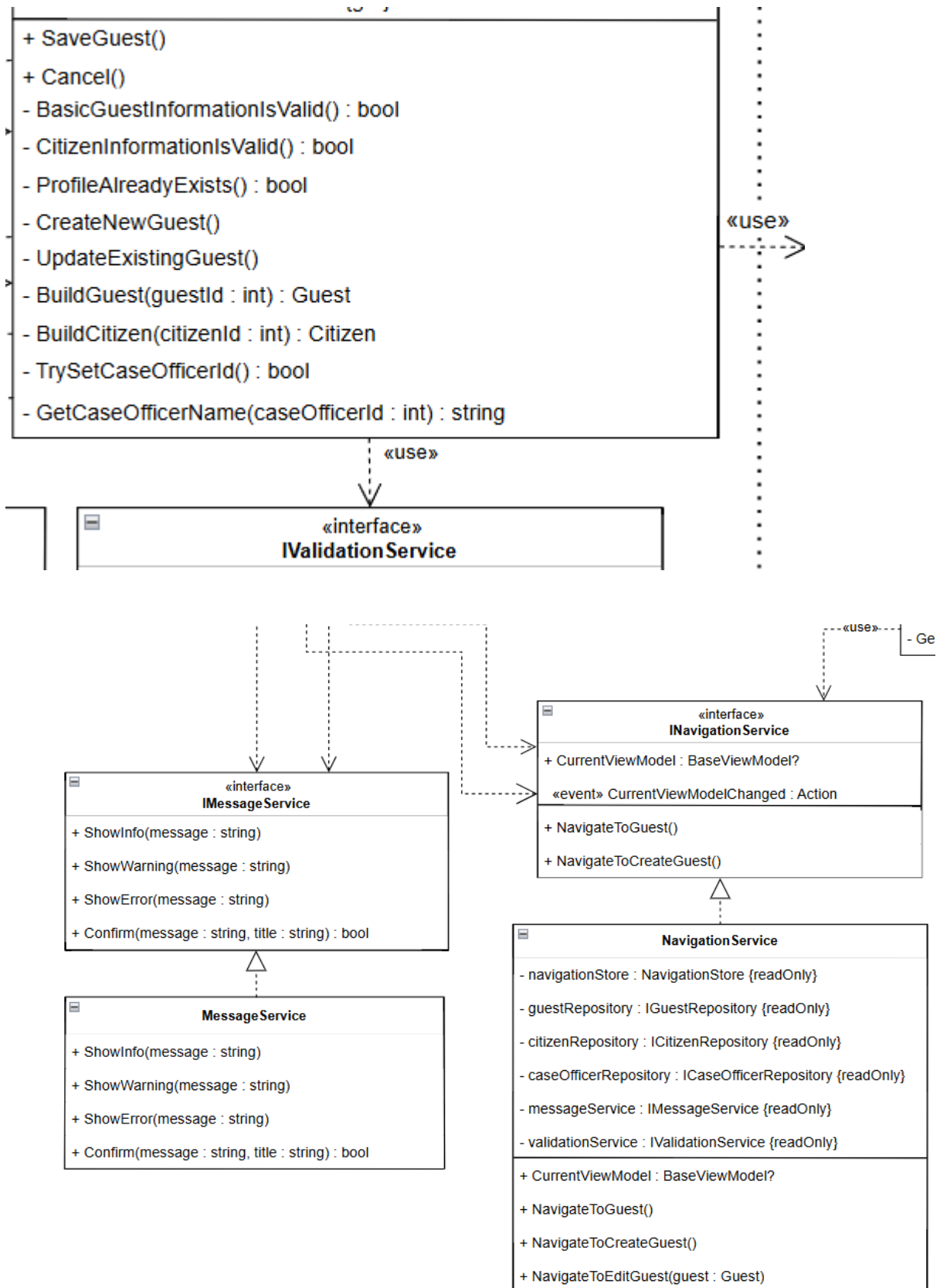
ViewModels

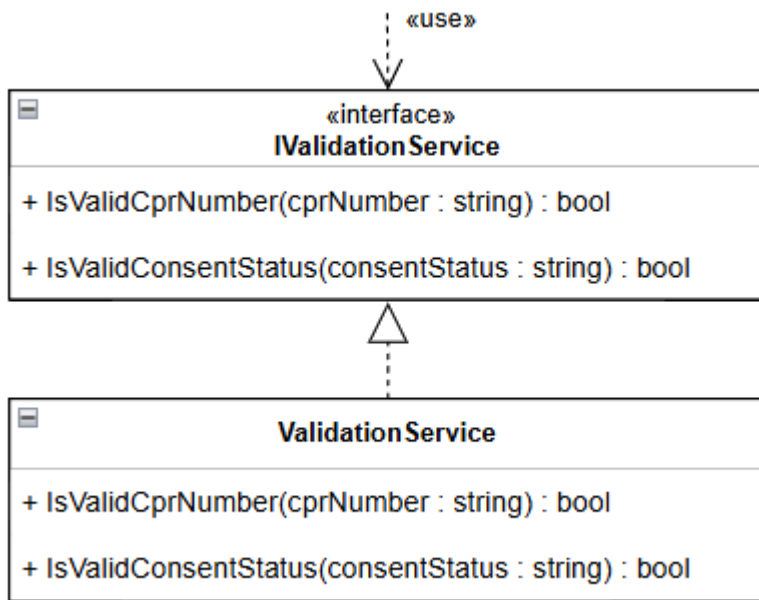


Application Layer (ViewModels, Service, Commands)

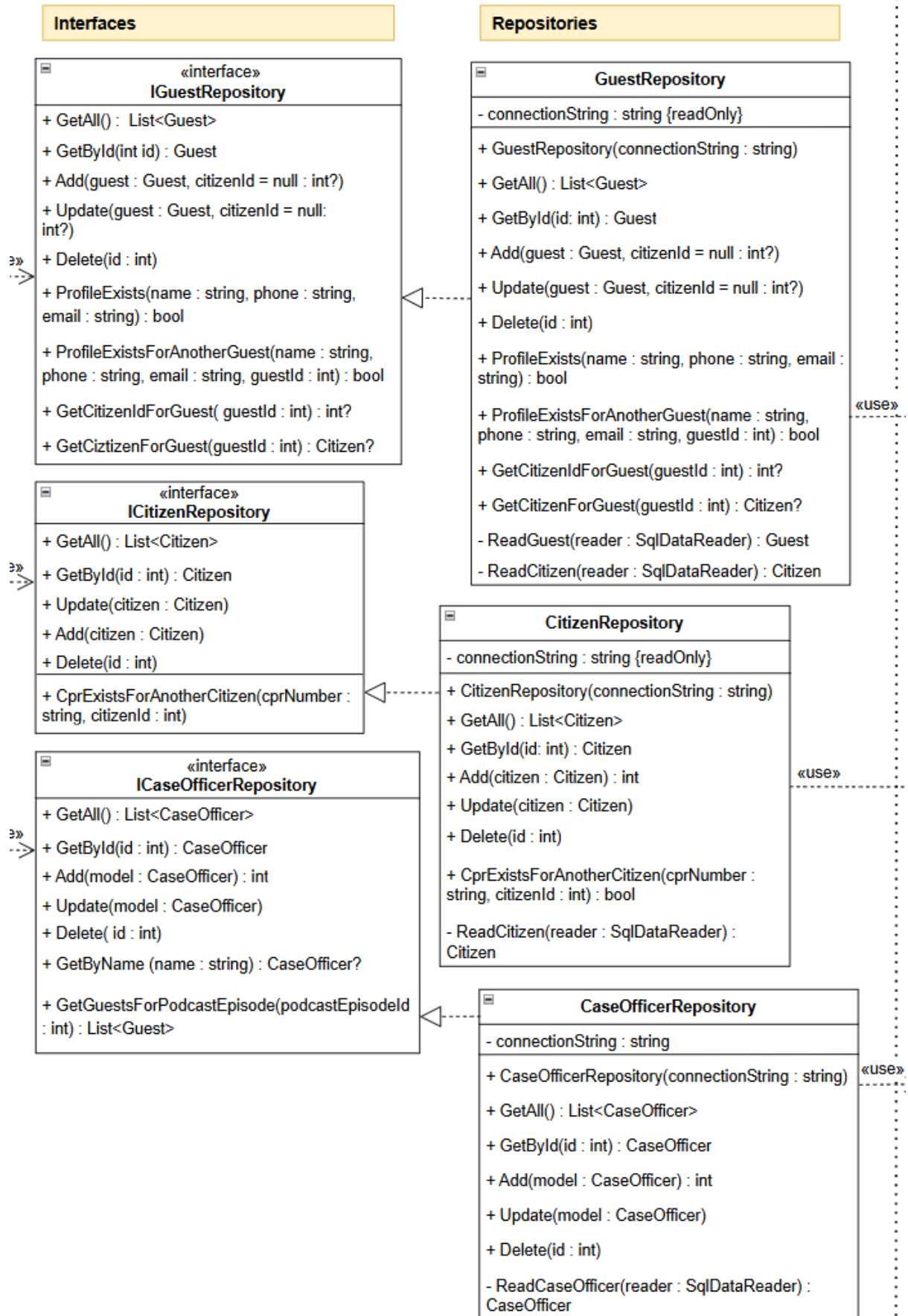


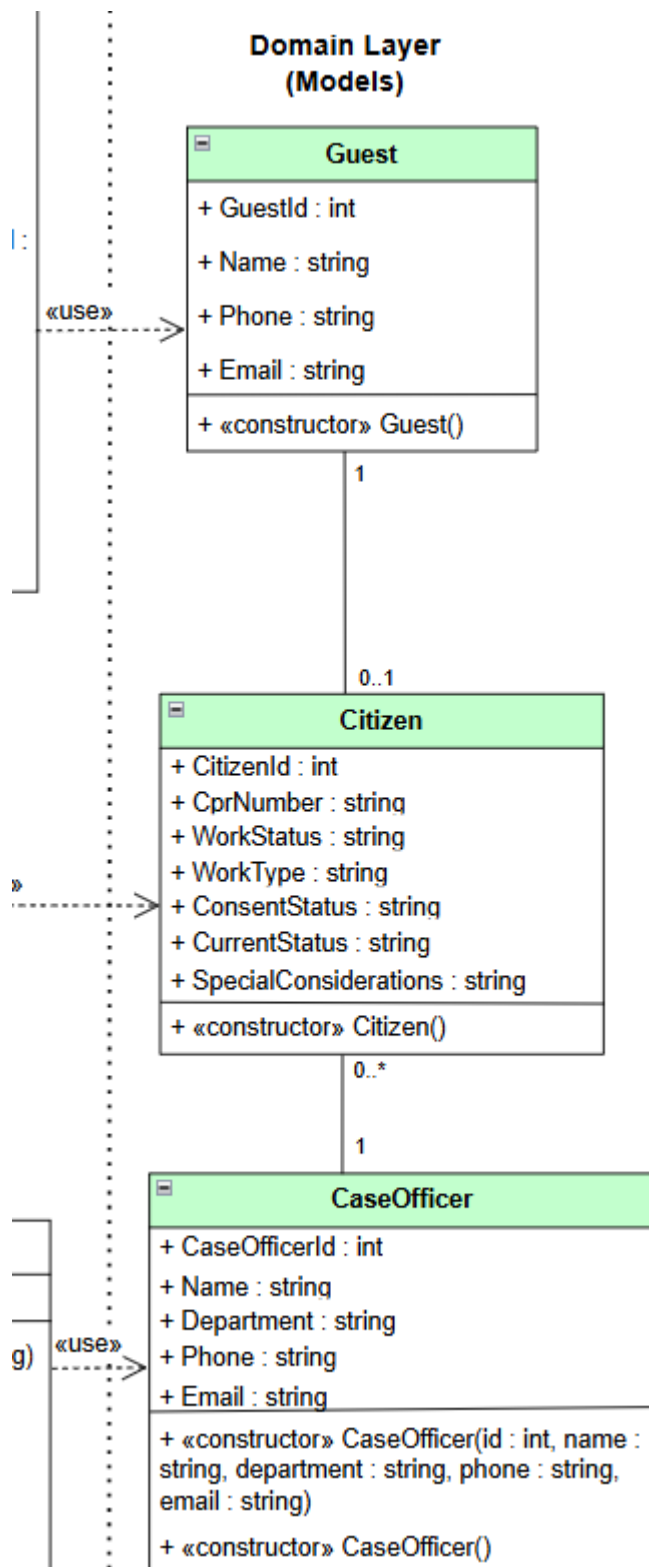






Persistence Layer (Repositories)





Figur 12: DCD for UC1: Opret profil til podcast episode.

Kvalitetskriterier for designklassediagrammet:

Metoder er formateret som operation (inputParametre : datatype) : returdatatype. Attributter har synlighed (+/-), attributNavn : datatype, og modellen har direkte sporbarhed til domæne- og objektmodel. Derudover er navngivningen gennemskuelig, og klasser er struktureret i kasser opdelt i klassenavn, attributter og operationer.

4.4 Sekvensdiagram

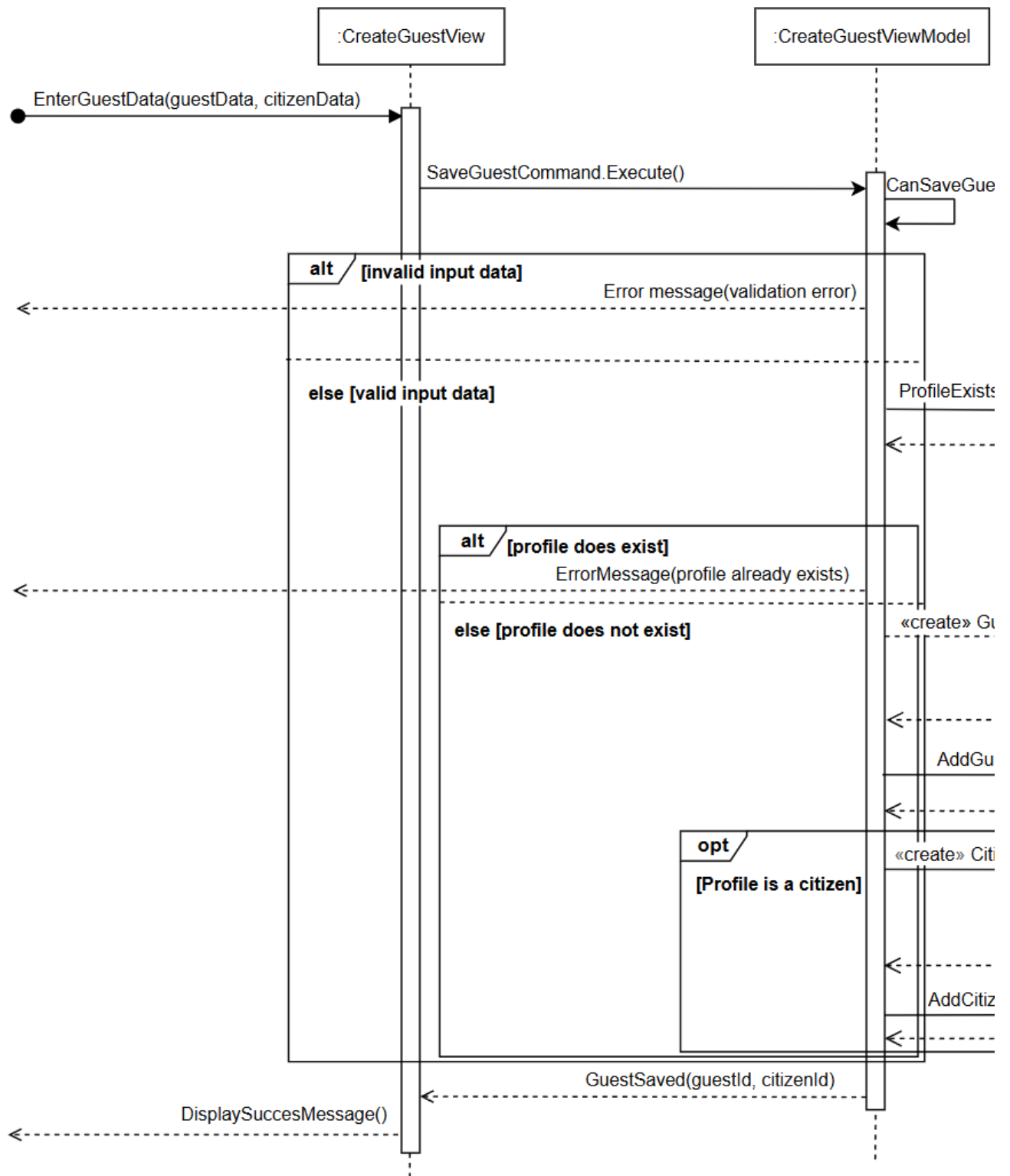
Vores Sekvensdiagram(SD) illustrerer vores use case, “Opret profil til podcast episode” og beskriver interaktionen mellem systemets objekter i forbindelse med oprettelsen af en gæsteprofil. Den viser den systematiske rækkefølge af beskeder mellem systemets komponenter. SD’et består af objekter som “:CreateGuestView” og “CitizenRepository”. Disse objekter er tilknyttede stiplede linjer, som kaldes for livslinjer. Langs livslinjerne kan man se aktivering bokse, som beskriver hvornår de individuelle objekter er i brug i systemet, hvis de udfører en operation eller behandler en besked.

Hele sekvensen starter med beskeden “EnterGuestData(guestData, citizenData)”. Denne besked sendes til “:CreateGuestView” objektet. Denne bliver derefter sendt til “:CreateGuestViewModel”. Dette gøres fordi at vores brugergrænseflade ikke selv håndterer systemets forretningslogik, dette bliver i stedet håndteret af dets viewmodel. Derefter bliver dataen valideret via “CanSaveGuest()” operationen. Herefter anvender vi “alt”, altså alternative fragmenter, hvor systemet checker om dataen, som sagt, er gyldig. Hvis dataen er ugyldig, så returneres en fejlmeddelelse. Hvis den er gyldig, så fortsætter sekvensen, og derefter checker om en profil med matchende data allerede eksisterer. Her kaldes operationen “ProfileExists(name, phone, email)” til “GuestRepository”. Repository’et returnerer dermed resultatet. Her bliver der anvendt et nyt alt fragment ligesom tidligere i sekvensen.

Hvis profilen eksisterer, så returneres en fejlbesked. Hvis profilen ikke allerede eksisterer, så

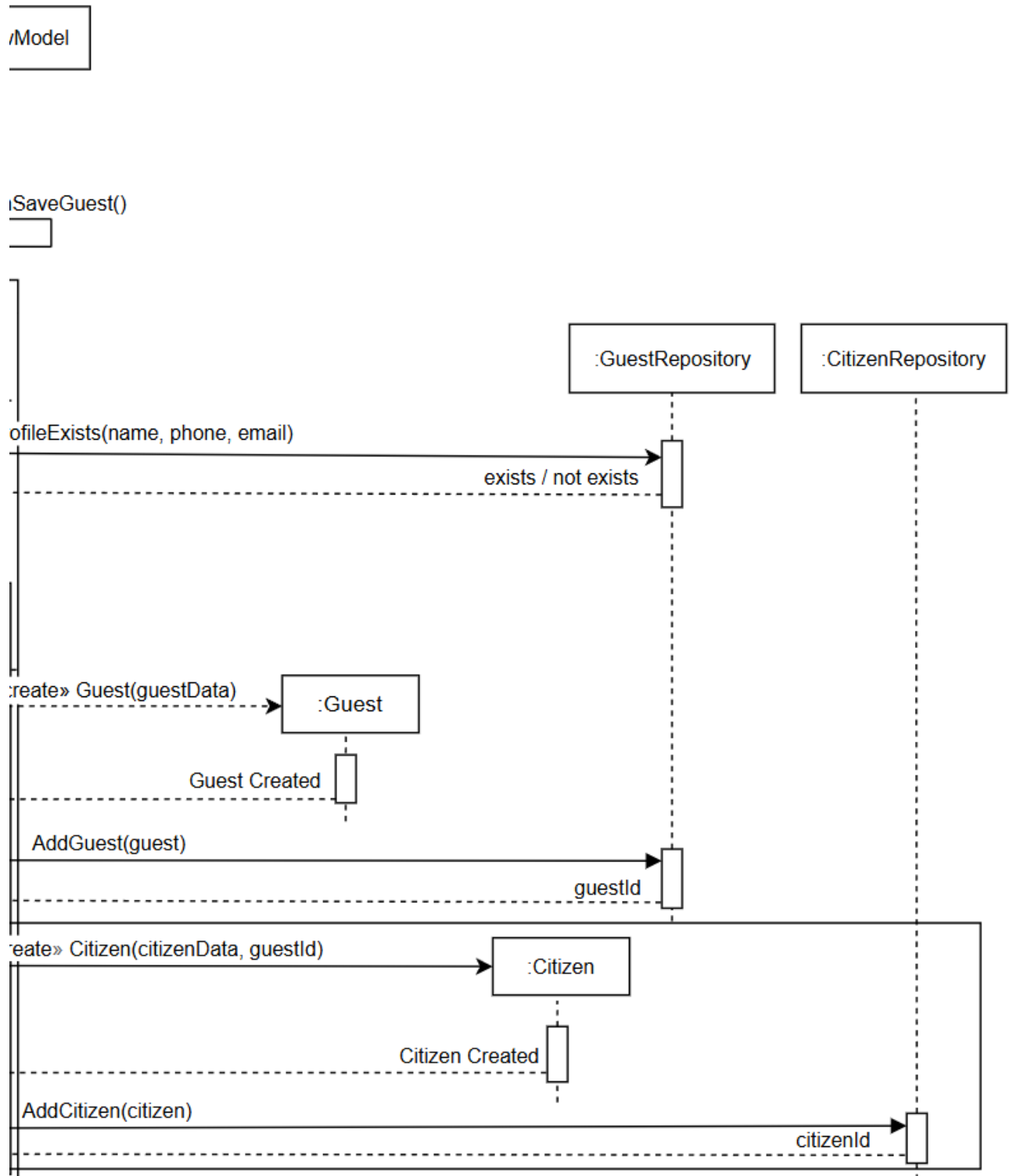
fortsætter sekvensen som normalt, og opretter et nyt domæneobjekt. “<<create>>
Guest(guestData)”. Objektet “:Guest” instantieres, hvorefter systemet returnerer “Guest
Created” Viewmodel’en kalder dermed “AddGuest(guest)” på objektets “:GuestRepository”
som persistere objektet og returnere et “guestId”.

Til sidst, udfører sekvensen et optional fragment. Fragmentet kører kun, hvis gæsten også
skal registreres og oprettes som borger. Når denne operation er gennemført, returnerer
ViewModel’en “GuestSaved(guestId, citizenId)” til “:CreateGuestView” objektet. Use Casen
afsluttes dermed ved at vise “DisplaySuccessMessage()” operationen til brugeren.



Figur 13 - Sekvensdiagram (del 1) for UC1.⁷

⁷ Tegnet i Draw.io



Figur 14 - Sekvensdiagram (del 2) for UC1.⁸

⁸ Tegnet i Draw.io

Kvalitetskriterier sekvensdiagrammet:

Beskederne beskriver tydeligt operationerne og er navngivet i overensstemmelse med metoderne. Relevante frames som alt, opt, og loops anvendes i henhold til use casen. Abstraktionsniveauet er konsistent gennem hele diagrammet. Aktører og systemkomponenter er klart navngivet, og stemmer overens med vores Domænemodel og DCD, og indholdet kan spores til en use case eller user story. Livslinjer samt pile er korrekt anvendt med tydelig oprettelse og nedlæggelse af objekter samt korrekt retning og notationer for messages og return messages

4.5 Databasedesign

4.5.1 Relationel Databaseskema

Vores relationelle databaseskema er opsat i tabeller, defineret med attributter og relationer mellem entiteterne. I vores database indgår der 5 tabeller som er opbygget af henholdsvis “GUEST”, “CITIZEN”, “CASEOFFICER”, “PODCASTEPISODE” og “PODCASTEPISODEGUESTS”.

Inden for hver tabel har vi henholdsvis de attributter, der understøtter hver tabel, men derudover har vi vores Primary Keys(PK) og Foreign Keys(FK).

“GUEST” har PK’en “GuestId” og FK’en “CitizenId”. Dette indikerer at en gæst kan være relateret til en registreret borger i vores system.

“CITIZEN” tabellen har PK’en “CitizenId” og FK’en “CaseOfficerId”.

“CASEOFFICER” har PK’en “CaseOfficerId” som anvendes og refereres i andre tabeller som en FK.

“PODCASTEPISODE” har PK’en “PodcastEpisodeId” og FK’en “CaseOfficerId”.

Til sidst har vi “PODCASTEPISODEGUESTS” som fungerer som en forbindelsestabel mellem “PODCASTEPISODE” og “GUEST”.

GUEST(GuestId, Name, Phone, Email, *CitizenId*)

CITIZEN(CitizenId, Name, Phone, Email, CPRNumber, WorkStatus, WorkType,
ConsentStatus, CurrentStatus, SpecialConsiderations, *CaseOfficerId*)

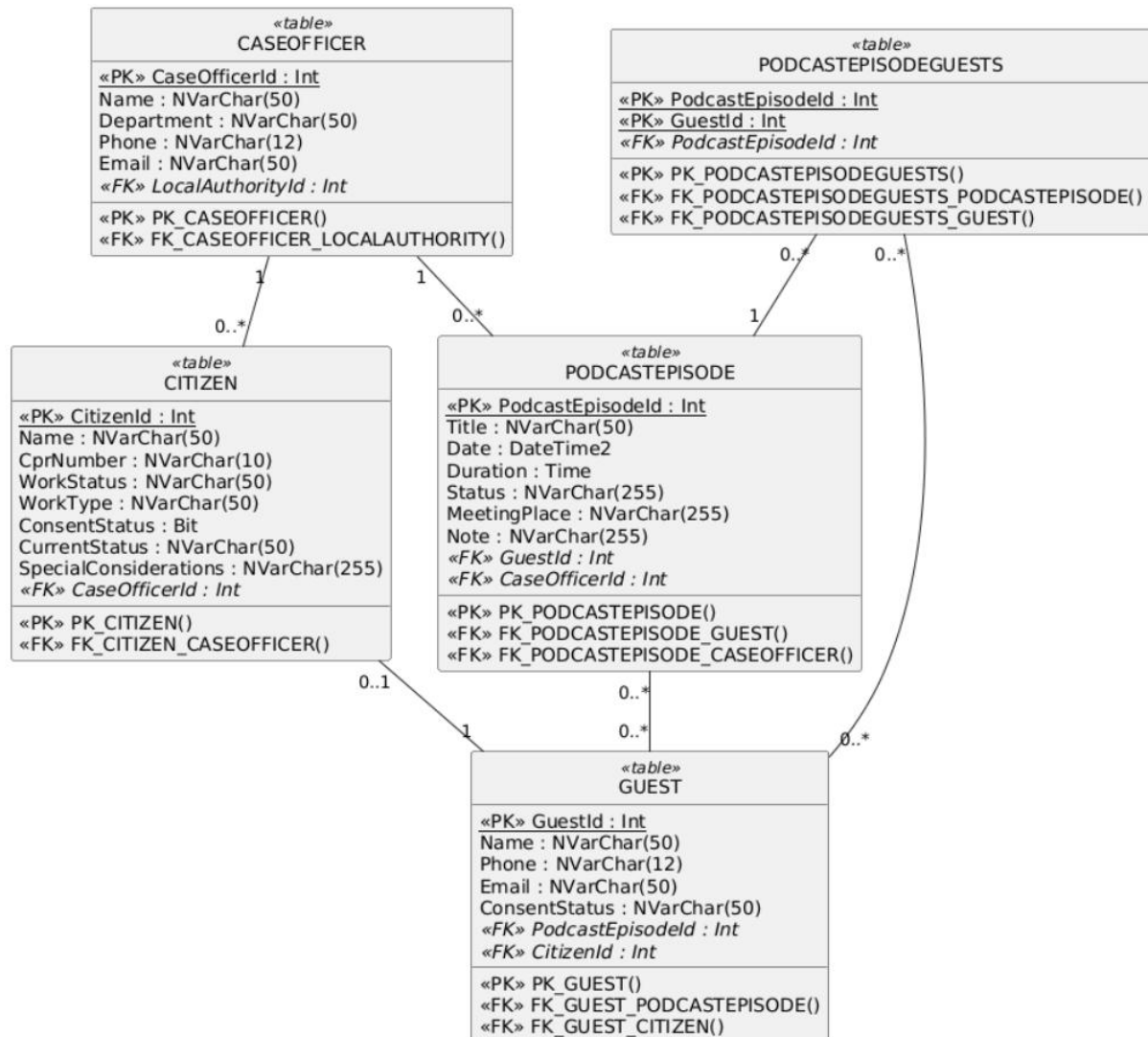
CASEOFFICER(CaseOfficerId, Name, Department, Phone, Email)

PODCASTEPISODE(PodcastEpisodeId, Title, Date, Duration, Status, MeetingPlace, Note,
CaseOfficerId)

PODCASTEPISODEGUESTS(PodcastEpisodeId, GuestId)

4.5.2 UML DatabaseModel

I denne model viser vi egentlig det samme som i den relationelle version, men dybere beskrevet, med relationer, datatyper og antallet af tegn som felterne må indeholde.



Figur 15 - UML Database Model.⁹

⁹ Tegnet i PlantUML

5. Implementering

Implementeringen af Soul & Talk er baseret på C#, WPF og SQL Server. Programmet er opdelt i flere lag, så brugergrænsefladen ikke selv står for databaseadgang eller forretningslogik. View-laget viser brugergrænsefladen, ViewModel-laget håndterer præsentrationslogik og brugerhandling, mens persistence-laget står for kommunikationen med databasen.

5.1 Implementeringsvalg

I projektet anvendes WPF til at opbygge brugergrænsefladen. WPF passer godt til projektet, fordi det understøtter databinding, commands og en tydelig opdeling mellem View og ViewModel.

Systemet er opbygget efter MVVM-arkitekturen. View er ansvarlig for brugergrænsefladen og binder til data fra ViewModel. Kun data og commands, gennem ViewModel, kan bruges af UI'et. Resten af systemets logik håndteres i de bagvedliggende lag.

Konkrete command-klasser anvendes til knapper og brugerhandling. Disse command-klasser arver fra BaseCommand, som implementerer ICommand fra WPF-frameworket. På den måde kan hver brugerhandling placeres i sin egen command-klasse, mens ViewModel indeholder den metode, som command'en kalder.

ViewModels anvender INotifyPropertyChanged, så ændringer i data automatisk kan opdateres i brugergrænsefladen. Det bruges eksempelvis, når inputfelter ift. oplysninger om gæst eller borger - ændrer sig i forbindelse med oprettelse eller redigering af en profil.

Oprettelse, ændring og sletning af data håndteres i ViewModel og sendes videre til persistence- og model-laget via CRUD-operationer (Create, Read, Update, Delete), som beskriver de grundlæggende måder at manipulere med data på i databasen.

5.2 C#-kode

Der vises dele af koden, som sætter fokus på vores UC1: Opret profil til podcastepisode.

5.2.1 GuestViewModel

```
48 public GuestViewModel(  
49     IGuestRepository guestRepository,  
50     INavigationService navigationService,  
51     IMessageService messageService)  
52 {  
53     _guestRepository = guestRepository;  
54     _navigationService = navigationService;  
55     _messageService = messageService;  
56  
57     Guests = new ObservableCollection<Guest>();  
58  
59     LoadGuestsCommand = new GuestLoadCommand(this);  
60     CreateGuestCommand = new GuestCreateNavigationCommand(this);  
61     EditGuestCommand = new GuestEditCommand(this);  
62     DeleteGuestCommand = new GuestDeleteCommand(this);  
63     BackCommand = new GuestBackNavigationCommand(this);  
64  
65     LoadGuests();  
66 }  
67  
68 2 references | Senan Salah, 5 days ago | 2 authors, 3 changes  
69 public void LoadGuests()  
70 {  
71     Guests.Clear();  
72  
73     foreach (Guest guest in _guestRepository.GetAll())  
74     {  
75         Guests.Add(guest);  
76     }  
77  
78     SelectedGuest = null;  
79 }
```

Figur 16 - Kodeudsnit for GuestViewModel.

Figur 12 viser et kodeudsnit fra GuestViewModel, som fungerer som ViewModel for gæsteoversigten. I constructoren modtager klasse de nødvendige dependencies {ReadOnly} i DCD'et) gennem interfaces, IGuestRepository, INavigationService og IMessageService. Det betyder, at GuestViewModel ikke selv opretter repository- eller service-klasser, men får dem tilført udefra. Dette giver en mere løs kobling mellem klasserne.

5.2.2 CreateGuestViewModel

```
12  ✓ public class CreateGuestViewModel : BaseViewModel
13      {
14          private readonly IGuestRepository _guestRepository;
15          private readonly ICitizenRepository _citizenRepository;
16          private readonly ICaseOfficerRepository _caseOfficerRepository;
17          private readonly INavigationService _navigationService;
18          private readonly IMessageService _messageService;
19          private readonly IValidationService _validationService;
20
21          public string Name { get; set; }
22          public string Phone { get; set; }
23          public string Email { get; set; }
24          public string CprNumber { get; set; }
25          public string WorkStatus { get; set; }
26          public string WorkType { get; set; }
27          public string ConsentStatus { get; set; }
28          public string CurrentStatus { get; set; }
29          public string SpecialConsiderations { get; set; }
30          public string CaseOfficerName { get; set; }
31
32          public ICommand SaveGuestCommand { get; }
33          public ICommand CancelCommand { get; }
```

Figur 17 - Kodeudsnit for CreateGuestViewModel.

I følgende kodeafsnit ses CreateGuestViewModel-klassen. Den indeholder de properties, som formularen binder til. Det gælder Navn, Telefon, Email og eventuelle borgeroplysninger. Klassen anvender repository-interfaces og services, så ViewModel ikke selv arbejder direkte med databasen. SaveGuestCommand og CancelCommand bruges til at håndtere brugerens handlinger fra view'et.

5.2.2.1 SaveGuest() metode i CreateGuestViewModel

```
135     public void SaveGuest()
136     {
137 >         if (!BasicGuestInformationIsValid())...
141
142 >         if (HasCitizenInfo && !CitizenInformationIsValid())...
146
147 >         if (HasCitizenInfo && !TrySetCaseOfficerId())...
151
152     if (!ProfileAlreadyExists())
153     {
154         try
155         {
156             if (_isEditMode)
157             {
158                 UpdateExistingGuest();
159             }
160             else
161             {
162                 CreateNewGuest();
163             }
164
165             _messageService.ShowInfo("Gæsten er gemt.");
166             _navigationService.NavigateToGuest();
167         }
168         catch (Exception ex)
169         {
170             _messageService.ShowError("Gæsten kunne ikke gemmes: " + ex.Message);
171         }
172     }
173     else
174     {
175         _messageService.ShowInfo("Der findes allerede en anden gæst med samme navn, telefon og email.");
176         return;
177     }
178 }
```

Figur 18 - Kodeudsnit for SaveGuest().

SaveGuest-metoden styrer gem handlingen i UC1. Først kontrolleres det, om de grundlæggende gæsteoplysninger, altså navn, tlf & email er udfyldt. I første if-statement kigger vi på vores alternative flow, og opfylder SD'et med, at den tjekker om profilen allerede eksisterer inden at den gemmer. Når profilen er gemt, vises en besked til brugeren, og systemet navigerer tilbage til gæsteoversigten. BasicGuestInformationIsValid() vises i næste kodeudsnit.

5.2.3 BasicGuestInformationIsValid : bool

```
180     private bool BasicGuestInformationIsValid()
181     {
182         if (string.IsNullOrEmpty(Name)
183             || string.IsNullOrEmpty(Phone)
184             || string.IsNullOrEmpty(Email))
185         {
186             _messageService.ShowWarning("Udfyld venligst navn, telefon og email.");
187             return false;
188         }
189
190         return true;
191     }
```

Figur 19 - Kodeudsnit for validering af grund-oplysninger(navn, tlf, email).

5.2.4 CitizenInformationIsValid : bool

```
193     private bool CitizenInformationIsValid()
194     {
195         if (string.IsNullOrEmpty(CprNumber)
196             || string.IsNullOrEmpty(WorkStatus)
197             || string.IsNullOrEmpty(WorkType)
198             || string.IsNullOrEmpty(ConsentStatus)
199             || string.IsNullOrEmpty(CurrentStatus)
200             || string.IsNullOrEmpty(CaseOfficerName))
201         {
202             _messageService.ShowWarning("Udfyld venligst CPR, jobstatus, arbejdstype, " +
203                 "samtykkestatus, nuværende status og sagsbehandler.");
204             return false;
205         }
206
207         if (!_validationService.IsValidCprNumber(CprNumber))
208         {
209             _messageService.ShowWarning("Angiv venligst CPR-nummer i formatet DDMMYY-XXXX.");
210             return false;
211         }
212
213         if (!_validationService.IsValidConsentStatus(ConsentStatus))
214         {
215             _messageService.ShowWarning("Samtykkestatus skal være Ja eller Nej.");
216             return false;
217         }
218     }
```

Figur 20 - Kodeudsnit for validering af borgeroplysninger.

Valideringen er placeret i ViewModel, fordi den hører til præsentationslogikken omkring formularen. De grundlæggende oplysninger kontrolleres direkte i ViewModel, mens mere specifik validering, som CPR-format og samtykkestatus, sendes videre til IValidationService. På den måde undgår ViewModel at indeholde al valideringslogik selv.

5.2.5 ValidationService : IValidationService

```
3  ✓      public class ValidationService : IValidationService
4          {
5  ✓      public bool IsValidCprNumber(string cprNumber)
6          {
7          if (string.IsNullOrEmpty(cprNumber)
8  ✓      || cprNumber.Length != 11 || cprNumber[6] != '-')
9          {
10         return false;
11     }
12
13     for (int i = 0; i < 6; i++)
14     {
15     ✓    if (!char.IsDigit(cprNumber[i]))
16         {
17         return false;
18     }
19     }
20
21     for (int i = 7; i < 11; i++)
22     {
23     ✓    if (!char.IsDigit(cprNumber[i]))
24         {
25         return false;
26     }
27     }
28
29     return true;
30 }
```

Figur 21 - Kodeudsnit for validering af CPR-nummer.

5.2.6 GuestRepository.Add

```
56     public int Add(Guest guest, int? citizenId = null)
57     {
58         using (SqlConnection connection = new SqlConnection(connectionString))
59         {
60             connection.Open();
61
62             SqlCommand cmd = new SqlCommand(
63                 "INSERT INTO Guest (Name, Phone, Email, CitizenId) " +
64                 "VALUES (@Name, @Phone, @Email, @CitizenId); " +
65                 "SELECT CAST(SCOPE_IDENTITY() AS int);",
66                 connection);
67
68             cmd.Parameters.Add("@Name", SqlDbType.NVarChar, 50).Value = guest.Name;
69             cmd.Parameters.Add("@Phone", SqlDbType.NVarChar, 20).Value =
70                 string.IsNullOrEmpty(guest.Phone) ? DBNull.Value : guest.Phone;
71             cmd.Parameters.Add("@Email", SqlDbType.NVarChar, 50).Value = guest.Email;
72             cmd.Parameters.Add("@CitizenId", SqlDbType.Int).Value =
73                 citizenId.HasValue ? citizenId.Value : DBNull.Value;
74
75             return Convert.ToInt32(cmd.ExecuteScalar());
76         }
77     }
78 }
```

Figur 22 - Kodeudsnit for metoden Add i GuestRepository.

GuestRepository.Add er en af de CRUD metoder vi bruger til at tilføje en gæst. Add metoden laver en connection til vores database, der tjekkes for et CitizenId, inden en gæst bliver tilføjet, hvis dette er tilfældet laves gæst om til en borger, hvis ikke der er borgeroplysninger angivet, vil gæst blive gemt i databasen som DBNull.

5.2.7 GuestRepository.GetById

```
40  ✓      public Guest GetById(int id)
41      {
42  ✓      using (SqlConnection connection = new SqlConnection(connectionString))
43      {
44          connection.Open();
45
46  ✓      SqlCommand cmd = new SqlCommand("SELECT GuestId, Name, Phone, " +
47          "Email FROM Guest " +
48          "WHERE GuestId = @GuestId", connection);
49
50          cmd.Parameters.Add("@GuestId", SqlDbType.Int).Value = id;
51
52  ✓      using (SqlDataReader reader = cmd.ExecuteReader())
53      {
54          return reader.Read() ? ReadGuest(reader) : new Guest();
55      }
56      }
57  }
```

Figur 23 - Kodeudsnit for metoden GetById for GuestRepository.

Der anvendes using statements til at erklære en SqlConnection og en SqlDataReader, hvilket sikrer korrekt disponering af ressourcer, så forbindelsen til databasen lukkes automatisk efter brug, hvilket forebygger “memory leaks”. Så laver vi en SqlCommand med en parametriseret SQL-sætning, og ved at benytte en parameter som her (@GuestId) i stedet for en direkte string, så beskytter vi mod SQL injection. Ved at bruge reader.Read() så kan man kalde en metode (ReadGuest), som hvis ja foretager mapping, hvor data fra databasen overføres til et Guest objekt. Er det derimod et nej, så returneres der et tomt new Guest() objekt.

5.2.8 GuestRepository.Update

```
79      public void Update(Guest guest, int? citizenId = null)
80      {
81          using (SqlConnection connection = new SqlConnection(connectionString))
82          {
83              connection.Open();
84
85              SqlCommand cmd = new SqlCommand(
86                  "UPDATE Guest SET Name = @Name, Phone = @Phone, Email = @Email, CitizenId = @CitizenId " +
87                  "WHERE GuestId = @GuestId",
88                  connection);
89
90              cmd.Parameters.Add("@Name", SqlDbType.NVarChar, 50).Value = guest.Name;
91              cmd.Parameters.Add("@Phone", SqlDbType.NVarChar, 20).Value =
92                  string.IsNullOrEmpty(guest.Phone) ? DBNull.Value : guest.Phone;
93              cmd.Parameters.Add("@Email", SqlDbType.NVarChar, 50).Value = guest.Email;
94              cmd.Parameters.Add("@CitizenId", SqlDbType.Int).Value =
95                  citizenId.HasValue ? citizenId.Value : DBNull.Value;
96              cmd.Parameters.Add("@GuestId", SqlDbType.Int).Value = guest.GuestId;
97
98              cmd.ExecuteNonQuery();
99          }
100      }
```

Figur 24 - Kodeudsnit for metoden Update i GuestRepository.

Metoden 'Update' opdaterer en gæst i databasen ved at åbne en SQL-forbindelse og udføre en update-command, som tager parametre. Den indsætter værdier fra et Guest-objekt (navn, telefon og e-mail) i databasen. Til sidst udføres kommandoen med ExecuteNonQuery(), som opdaterer databasen uden at returnere data, eftersom metoden ikke har en returtype.

5.2.9 GuestRepository.Delete

```
102      public void Delete(int id)
103      {
104          int? citizenId = GetCitizenIdForGuest(id);
105
106          using (SqlConnection connection = new SqlConnection(connectionString))
107          {
108              connection.Open();
109
110              SqlCommand deleteEpisodeLinks = new SqlCommand(
111                  "DELETE FROM PodcastEpisodeLinks WHERE GuestId = @GuestId", connection);
112              deleteEpisodeLinks.Parameters.Add("@GuestId", SqlDbType.Int).Value = id;
113              deleteEpisodeLinks.ExecuteNonQuery();
114
115              SqlCommand deleteGuest = new SqlCommand("DELETE FROM Guest WHERE GuestId = @GuestId", connection);
116              deleteGuest.Parameters.Add("@GuestId", SqlDbType.Int).Value = id;
117              deleteGuest.ExecuteNonQuery();
118
119              if (citizenId.HasValue)
120              {
121                  SqlCommand deleteCitizen = new SqlCommand("DELETE FROM Citizen WHERE CitizenId = @CitizenId", connection);
122                  deleteCitizen.Parameters.Add("@CitizenId", SqlDbType.Int).Value = citizenId.Value;
123                  deleteCitizen.ExecuteNonQuery();
124              }
125          }
126      }
```

Figur 25 - Kodeudsnit for metoden Delete i GuestRepository.

Figur 16 viser hvordan der fjernes data fra databasen. Det kan vi se ved at vi har kaldt metoden `Delete(int id)`, for at beskrive hvad denne metode gør. Først får vi fat i data på gæsten igennem `GetCitizenIdForGuest`, derefter tager vi kontakt til sql databasen igennem vores `using` blokken, som betyder at den automatisk lukker og frigiver data, når koden er færdig med at være i brug. Det betyder altså at den først er færdig når alt koden som står i Tuborg klammerne fra `using` til under `if`-statementet at den først der frigives. Alt koden i disse tuborg klammer beskriver igennem vores `SqlCommand` at der fjernes data i diverse tabeller fra databasen. Der kan læses at der fjernes gæst fra podcast episode hvis gæsten er tilkoblet en episode, derefter fjernes gæsten under tabellen som indeholder en gæsteliste. Så i `if`-statementet fjernes data fra `Citizen` tabellen, hvis gæsten er sat til at have data på en borger.

Kvalitetskriterier for C#-kode:

CRUD-relateret logik er opdelt mellem `ViewModels`, `services` og `repositories`. `ViewModels` håndterer præsentrationslogik, `services` anvendes til funktioner, som navigation, beskeder og validering, mens `repositories` indkapsler dataadgang til databasen. Dette giver en overskuelig arkitektur, hvor de enkelte klasser har tydelige ansvarsområder.

I løsningen er der gjort brug af navnekonvention, CRUD-mønsteret, objektorienteret programmering, SOLID-princippet: Single responsibility principle samt GRASP-princippet om lav kobling og høj samhørighed fordi systemets dele er opdelt efter ansvar og kun afhænger af hinanden, hvor det er nødvendigt.

6. Test

6.1 Unit test

Unit test til UseCase 1:

```
4      namespace UseCase1_Test
5      {
6          [TestClass]
7          // 0 references | Senan Salah, 3 hours ago | 2 authors, 3 changes
8          public class Test1
9          {
10             [TestMethod]
11             // 0 references | Senan Salah, 3 hours ago | 2 authors, 3 changes
12             //Test for at teste om en Guest med borgeroplysninger er initialiseret korrekt.
13             public void Constructor_ShouldInitializePropertiesCorrectly_WhenValidDataIsProvided()
14             {
15                 // Arrange
16                 int expectedId = 1;
17                 string expectedName = "Mads Jensen";
18                 string expectedPhone = "12345678";
19                 string expectedEmail = "mads@example.dk";
20                 string expectedCpr = "123456-7890";
21                 string expectedWorkStatus = "Ledig";
22                 string expectedWorkType = "Kontor";
23                 string expectedConsent = "Givet";
24                 string expectedStatus = "Aktiv";
25                 string expectedNotes = "Ingen særlige hensyn";
26                 // Act
27                 var citizen = new Citizen(
28                     expectedId,
29                     expectedName,
30                     expectedPhone,
31                     expectedEmail,
32                     expectedCpr,
33                     expectedWorkStatus,
34                     expectedWorkType,
35                     expectedConsent,
36                     expectedStatus,
37                     expectedNotes,
38                     1
39                 );
40                 // Assert
41                 Assert.AreEqual(expectedId, citizen.CitizenId);
42                 Assert.AreEqual(expectedName, citizen.Name);
43                 Assert.AreEqual(expectedCpr, citizen.CprNumber);
44                 Assert.AreEqual(expectedWorkStatus, citizen.WorkStatus);
45                 Assert.AreEqual(expectedStatus, citizen.CurrentStatus);
46             }
47         }
48     }
```

Figur 26 - Kodeudsnit for Unit Test til UC1.

Vi har undervejs gjort brug af unit tests for at sikre os, at systemets klasser og funktionaliteter virker som forventet. Formålet med disse unit tests er at sikre korrekt initialisering af objekter og validerer at data håndteres korrekt i vores system. I denne unit test gør vi brug af Arrange, Act, Assert, for at tjekke om en Citizen instans er initialiseret korrekt med de angivne borgeroplysninger. Under “Arrange” opstiller vi testdata og initialisere det. Dette er værdier for borgerens oplysninger som navn, CPR nummer osv.

Under “Act” oprettes der en instans af klassen Citizen med disse angivne værdier.

Under “Assert” tjekker vi og sammenligner disse værdier med de faktiske værdier ved brug af “Assert.AreEqual()”. Det verificeres, at objektet er initialiseret korrekt og at klassens constructor fungerer, som vi forventer.

Kvalitetskriterier for Unit Test:

Testmetoden er opbygget efter AAA-Mønstret (Arrange, Act, Assert) for at sikre en overskuelig struktur, og metoderne skal have et sigende navn, som beskriver formålet med testen.

7. Projektstyring og arbejdsproces

Der blev oprettet sprint henover hele forløbet for at vise teamets forløb samt for at sætte nogle målsætninger for hver sprint. For at alle i teamet havde et fælles mål og forstod hinanden, lavede vi også en definition af hver sprint.

7.1 Kommunikation

Gruppens kommunikation foregik primært gennem Discord, Her oprettede vi kanaler til generel kommunikation, GitHub, UML-arbejde og rapportskrivning.

7.2 Versionsstyring og arbejdsmetode

Vi anvendte versionsstyring til at håndtere ændringer i koden samt dokumentation gennem Github. Det gjorde det nemmere at arbejde fælles omkring koden, men også for at samle ændringer i vores projektmappe gennem VSCode. Det hjalp med at skabe et overblik over, hvem der har pushet hvilke ændringer, i projektet eller i programmet. Man har også muligheden for at gå tilbage til tidligere versioner hvis der skulle ske fejl.

Vi har gjort brug af GitHub samt Git/Git Bash. Github er brugt til at holde vores kode afsnit samlet, holde styr på hvem der har pushet hvad, og muligheden for at oprette forskellige branches vi kunne arbejde på samtidig, uden at støde ind i konflikter. Git er brugt til at pulle vores kode og opdateringer ned til vores individuelle systemer. Samt pushe vores kode ud til resten af holdet, gennem Github.

Vores arbejdsmetode har primært været ved brug af to forskellige branches. Vi har haft en main branch, samt en view branch. Da vores system ikke er større end det er, og da der er blevet aftalt hvem der tager sig af bestemte dele af koden, har det ikke været et stort problem med kun to branches. Men det ville stadig have været en god ide, og mere failproof, at have en branch for hvert medlem, hvor man frit kan arbejde uden at være bange for at pushe en opdatering som går ind og roder med andres kode. Selvom vi kun har haft brug af to branches, har vi formået at undgå nogle store konflikter, når der skulle committes.

Som refleksion, ville det helt sikkert være best practice at oprette branches for hvert gruppemedlem. Så ville vi være langt mere sikret mod eventuelle konflikter, når der skulle pushes kode. Vi kunne eventuelt også have gjort mere brug af pull-requests, før vi har merget branches. Dette tillader nemlig, at vi meget nemmere ser ændringerne inde i Github samt giver mulighed for at diskutere og reviewe kode, inden det bliver merget.

GitHub har også været til stor hjælp, i forhold til sporbarhed af commits, samt stor hjælp til at skabe overblik over hvilke ændringer der er blevet lavet på bestemte tidspunkter. Hvilket bidrog til en langt mere struktureret arbejdsproces.

7.3 Projektstyring

Sprints

Pre-sprint:

I pre-sprintet prioriterede vi at styrke samarbejdet i gruppen samt at forsøge at sikre en stabil arbejdsgang ved hjælp af en gruppekontrakt. Udover dette havde vi fokus på at kommunikere med PO og forventningsafstemme, så vi havde et mere klart blik for POs ønsker.

Sprint 1 - uge 15:

I vores første sprint valgte vi at fokusere på analysedelen, hvilket indebar modellering af et første udkast af nedenstående analyseartefakter:

- Use Cases & Use Case Diagram
- Objektmodel
- Systemsekvensdiagram
- Systemoperationskontrakter
- BPMN
- Arbejde med Scrum
- Opstart på systemdokumentation
- Business Case
- Interessentanalyse

Efter udførelsen af sprintet afholdte vi et sprint-retrospective meeting, hvor vi gennemgik de udarbejdede artefakter. Da processen er iterativ, var vi forberedte på senere at være nødsaget til at vende tilbage for at justere artefakterne.

Vores 'Definition of done' for dette sprint indebar blot, at artefakterne var udarbejdet.

Sprint 2 - uge 16:

I vores anden sprint havde vi fokus på at udarbejde flere artefakter samt at klargøre implementeringsdelen. Vores fokus lå på nedenstående artefakter:

- Wireframe
- DCD (MVVM)
- Pakkediagram
- SD
- Relationel Databasemodel
- UML Databasemodel

Under udførelsen af dette sprint endte vi også med at justere i de artefakter, vi udarbejdede i forrige sprint. Vi ændrede først nogle klasser i domænemodellen, og derefter genbesøgte vi de resterende artefakter.

Hen imod slutningen af ugen mødtes vi med vores sparringsteam for at afholde det første sparringsmøde. Vi modtog brugbar feedback og rettede derefter til i slutningen af sprintet. Vores 'Definition of done' bestod af, at vi nåede det, vi satte os for. Derudover fik vi feedback fra undervisere og implementerede disse. Vi havde hovedsageligt fokus på designdelen i dette sprint.

Sprint 3 - uge 17-18:

I vores tredje sprint havde vi ambitioner om at udarbejde en SQL database model samt at implementere de forskellige lag i C#:

- SQL Databasemodel
- C# implementering (Domain - Application/Persistence layer - View layer)

I dette sprint brugte vi dog meget tid på at genbesøge vores tidligere udarbejdede artefakter. I samarbejde med PO nåede vi nemlig frem til forskellige dele, som var unødvendige for det kommende system. Af den grund ændrede vi fokus fra at implementere kode til at finpudse artefakter før implementeringen omkring halvvejs inde i sprintet.

‘Definition of done’ i dette sprint blev justeret til, at vi delvist havde implementeret kode. Vi endte med at udskyde den færdige implementering til sprint nummer fire. I vores retrospective meeting nåede vi frem til, at vi muligvis havde været en smule urealistiske, men at vi stadig havde tid til at nå i mål.

Sprint 4 - uge 19-20:

I vores fjerde og sidste sprint har vi haft fokus på finpudsning af kode samt kvalitetssikring i forhold til artefakter. Derudover har vi oprettet en unittest, som tester, at programmet lever op til formålet med use case 1. Den sidste del af tiden har vi brugt på at skrive rapport.

- C# implementering (View - ViewModel layer)
- Unittest (Automatiseret)

8. Konklusion

Gennem dette projekt har vi udviklet et administrativt system til Soul & Talk, der forsøger at løse de identificerede udfordringer med manglende struktur og manuel planlægning af podcastproduktion. Ved at erstatte spredte notater og manuelle processer med en centraliseret løsning, har vi skabt fundamentet for en mere effektiv arbejdsgang i virksomheden.

Vi har opnået følgende resultater:

- **Systematisk styring:** Med implementeringen af CRUD-operationer kan Soul & Talk nu administrere gæster, borgeroplysninger og podcast-episoder i én samlet database, hvilket sikrer datakonsistens og sporbarhed.
- **Optimeret brugerflade:** Gennem arbejdet med wireframes og MVVM-arkitekturen har vi udviklet en brugerflade i WPF, der er intuitiv for product owner og adskiller logik fra design.
- **Faglig dokumentation:** Projektet dokumenterer hele processen fra den indledende domænemodel til den endelige tekniske implementering, hvilket gør det muligt for fremtidige udviklere at vedligeholde og skalere systemet.

Vi konkluderer, at systemet opfylder de opstillede krav i kravspecifikationen og imødekommer PO's behov for en mere struktureret podcastadministration. Systemet leverer den nødvendige funktionalitet til at professionalisere arbejdet i Soul & Talk og understøtter en mere effektiv håndtering af gæster, borgere og podcast-episoder. Dermed fungerer systemet ikke kun som et teknisk produkt, men også som et praktisk værktøj, der skaber forretningsmæssig værdi.

9. Litteraturliste

- Underviseres uddelte materiale
- Craig Larman. (2004). Applying UML and Patterns 3. Udg.
- Rob Miles. (2019). C# Programming Yellow Book. Cheese Edition 8.1

10. Bilag

10.1 Bilag 1: GitHub repository

Link: https://github.com/PeterZimmertoft/Team_Soul_-_Talk-PodcastProgram

10.2 Bilag 2: UML → SQL-Query

```
1  CREATE TABLE CaseOfficer
2  (
3      CaseOfficerId INT IDENTITY(1,1) NOT NULL,
4      Name NVARCHAR(50) NOT NULL,
5      Department NVARCHAR(50) NOT NULL,
6      Phone NVARCHAR(20) NOT NULL,
7      Email NVARCHAR(50) NOT NULL,
8
9      CONSTRAINT PK_CaseOfficer PRIMARY KEY (CaseOfficerId),
10 );
```

```
12 CREATE TABLE Citizen
13 (
14     CitizenId INT IDENTITY(1,1) NOT NULL,
15     Name NVARCHAR(50) NOT NULL,
16     Phone NVARCHAR(20) NULL,
17     Email NVARCHAR(50) NOT NULL,
18     CprNumber NVARCHAR(11) NOT NULL,
19     WorkStatus NVARCHAR(50) NOT NULL,
20     WorkType NVARCHAR(50) NOT NULL,
21     ConsentStatus BIT NOT NULL,
22     CurrentStatus NVARCHAR(50) NOT NULL,
23     SpecialConsiderations NVARCHAR(255) NULL,
24     CaseOfficerId INT NULL,
25
26     CONSTRAINT PK_Citizen PRIMARY KEY (CitizenId),
27     CONSTRAINT UQ_Citizen_CprNumber UNIQUE (CprNumber)
28     CONSTRAINT FK_Citizen_CaseOfficer
29     FOREIGN KEY (CaseOfficerId) REFERENCES CaseOfficer(CaseOfficerId)
30 );
```

```

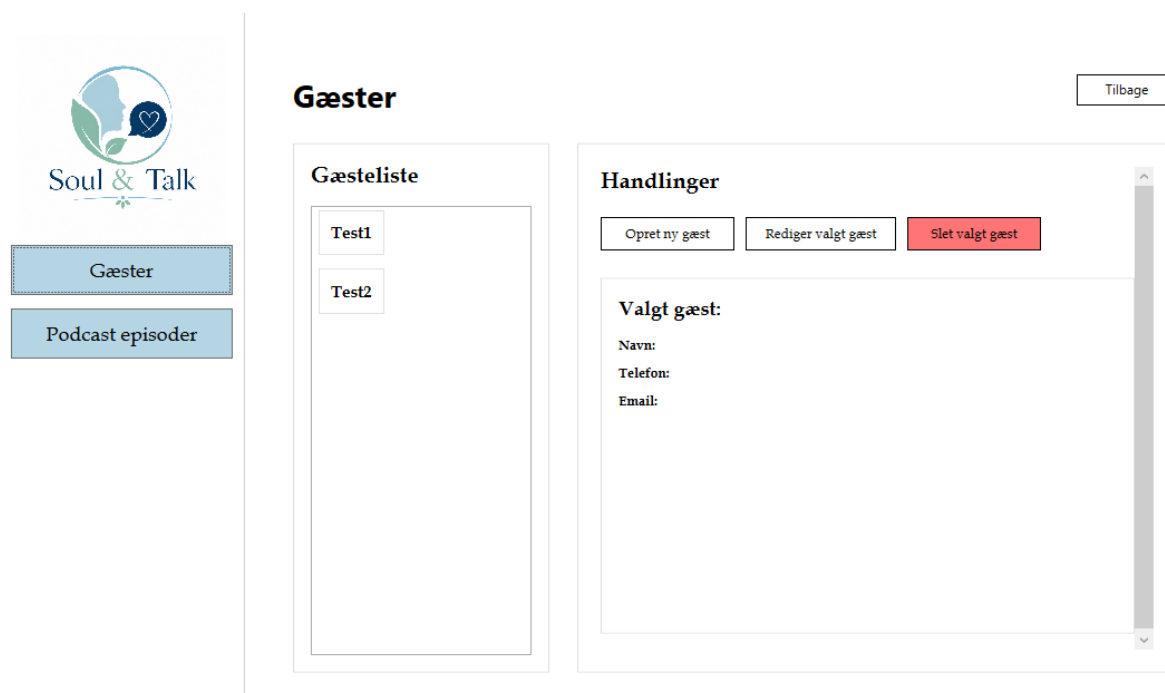
32 CREATE TABLE Guest
33 (
34     GuestId INT IDENTITY(1,1) NOT NULL,
35     Name NVARCHAR(50) NOT NULL,
36     Phone NVARCHAR(20) NULL,
37     Email NVARCHAR(50) NOT NULL,
38     CitizenId INT NULL,
39
40     CONSTRAINT PK_Guest PRIMARY KEY (GuestId),
41     CONSTRAINT FK_Guest_Citizen
42     FOREIGN KEY (CitizenId) REFERENCES Citizen(CitizenId)
43 );
44
45 CREATE TABLE PodcastEpisode
46 (
47     PodcastEpisodeID INT IDENTITY(1,1) NOT NULL,
48     Title NVARCHAR(100) NOT NULL,
49     [Date] DATETIME2 NOT NULL,
50     Duration INT NOT NULL,
51     Status NVARCHAR(50) NOT NULL,
52     MeetingPlace NVARCHAR(100) NOT NULL,
53     Note NVARCHAR(255) NULL,
54     CaseOfficerId INT NOT NULL,
55
56     CONSTRAINT PK_PodcastEpisode PRIMARY KEY (PodcastEpisodeID),
57     CONSTRAINT FK_PodcastEpisode_CaseOfficer
58     FOREIGN KEY (CaseOfficerId) REFERENCES CaseOfficer(CaseOfficerId)
59 );
60
61 CREATE TABLE PodcastEpisodeGuests
62 (
63     PodcastEpisodeID INT NOT NULL,
64     GuestId INT NOT NULL,
65
66     CONSTRAINT PK_PodcastEpisodeGuests PRIMARY KEY (PodcastEpisodeID, GuestId),
67     CONSTRAINT FK_PodcastEpisodeGuests_PodcastEpisode
68     FOREIGN KEY (PodcastEpisodeID) REFERENCES PodcastEpisode(PodcastEpisodeID),
69     CONSTRAINT FK_PodcastEpisodeGuests_Guest
70     FOREIGN KEY (GuestId) REFERENCES Guest(GuestId)
71 );

```

10.3 Bilag 3: MainView (hovedvinduet i programmet)



10.3.1 Bilag 3.1: GuestView (Gæstevinduet i programmet)



10.4 Bilag 3.2: CreateGuestView (Opret gæsteprofil i programmet)



Gæster

Podcast episoder

Opret gæsteprofil

Indtast gæstens oplysninger. Hvis gæsten også er borger, kan ekstra informationer udfyldes.

☐ Tilføj borgeroplysninger

Basisoplysninger

Navn

Telefon

Email

Borgeroplysninger

CPR-nummer (DDMMYY-XXXX)

Jobstatus



Gæster

Podcast episoder

CPR-nummer (DDMMYY-XXXX)

Jobstatus

Arbejdstype

Samtykkestatus

Nuværende status (fx Aktiv/Passiv)

Sagsbehandler

Særlige hensyn

Gem profil

Annuller

10.5 Bilag 4: Unit Test

```
5      [TestClass]
6      0 references | Tore, 22 hours ago | 1 author, 1 change
7      v public class GuestRepositoryTests
8      {
9      10      private readonly string _connectionString =
10      "Server=localhost;Database=SoulTalkDB;Trusted_Connection=True;TrustServerCertificate=True;";
11      private GuestRepository _repository;
12
13      [TestInitialize]
14      0 references | Tore, 22 hours ago | 1 author, 1 change
15      v public void Setup()
16      {
17      _repository = new GuestRepository(_connectionString);
18      }
19
20      [TestMethod]
21      0 references | Tore, 22 hours ago | 1 author, 1 change
22      v public void Add_And_GetById_ShouldPersistAndReadGuest()
23      {
24      // Arrange
25
26
27      // Arrange
28
29      var testGuest = new Guest
30      {
31      Name = "Senan Bradley",
32      Phone = "1234567890",
33      Email = "Senan@Bradley.com",
34      };
35
36      // Act
37
38      int newGuestId = _repository.Add(testGuest);
39
40      Guest retrievedGuest = _repository.GetById(newGuestId);
41
42      // Assert
43
44      Assert.IsNotNull(retrievedGuest);
45      Assert.AreEqual(testGuest.Name, retrievedGuest.Name);
46      Assert.AreEqual(testGuest.Phone, retrievedGuest.Phone);
47      Assert.AreEqual(testGuest.Email, retrievedGuest.Email);
48      }
49      }
```


10.6 Bilag 5: Komplet DCD for UC1

